

NetVis: A Composable Architecture for Large-Scale Network Visualization

Technical Reference

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.5.0

Last updated: March 13, 2026

Abstract. NetVis is a Rust-based network topology layout and visualization engine comprising 90,000+ lines of code. It transforms complex multi-layer network topologies into publication-quality visualizations through four contributions: (1) a dual-engine force-directed layout with Barnes–Hut acceleration for 10,000+ node topologies; (2) R*-tree-accelerated label placement achieving $O(\log n)$ collision queries; (3) a composable refiner pipeline separating layout from visual polish; and (4) sixteen automated quality metric families enabling regression testing. This document is the definitive technical reference for NetVis. It describes the data model, architecture, nine layout engines, ten edge routing and post-layout refiners, parallel edge rendering (five modes with automatic LAG/bond aggregation), interface rendering, node shape library, edge gradients, path emphasis, metric encoding, rendering pipeline, topology diffing, temporal analysis, traffic animation, annotation markup, interactive editing, visual effects, graph analytics, filter DSL, 16 quality metric families, and public API. Appendices provide the full YAML topology schema, performance baselines with comparative benchmarks against Graphviz, and a complete configuration reference.

Contents

1	Introduction	1
1.1	Design Philosophy	2
1.2	Output Gallery	2
1.3	Audience	3
1.4	How to Read This Document	3
2	NetVis by Example	3
2.1	Basic Campus Network	3
2.2	Layout Algorithm Comparison	3
2.3	Geographic CDN	4
2.4	Isometric Multi-Layer	4
2.5	Parallel Edges and Bond Groups	5
2.6	Subway Layout	5
2.7	Topology Diff and Analysis	6
3	Data Model	6
3.1	Node and Edge Data	7
3.2	Edge Groups	7
3.3	Scene Graph	7
4	Architecture	8
4.1	Pipeline Composition	8
4.2	Scene Graph as Intermediate Representation	9
5	Layout Algorithms	9
5.1	Force-Directed Layout	9
5.2	Hierarchical Layout (Sugiyama)	9
5.3	Geographic Layout	10
5.4	Isometric Multi-Layer Layout	10
5.5	Orthogonal Layout	11
5.6	Subway Layout	11
5.7	Radial Tree, Tree, and Composite	11
6	Theme Gallery	12
7	Parallel Edge Rendering	13
7.1	Parallel Edge Mode Decision Tree	14
7.2	Automatic Bandwidth Aggregation	14
8	Node Shapes and Device Icons	14
8.1	Available Shapes and Icons	14
8.2	Multi-Part Semantic Rendering	15
9	Interface Rendering	16
10	Edge Gradients and Path Emphasis	17
10.1	Edge Gradients	17
10.2	Path Emphasis	17
11	Metric Encoding	18
12	Label Placement	19
12.1	Problem and Algorithm Choice	19
12.2	Two-Phase Greedy Algorithm	19
12.3	Candidate Scoring Visualization	20
12.4	Performance	20
13	Edge Bundling	20
13.1	FDEB Force Model	21
14	Edge Routing and Post-Layout Refiners	22
14.1	Smart Routing: Visibility Graph	23
14.2	Edge Clipping: Before and After	23
14.3	Deoverlap: Before and After	23
14.4	Boundary Bundling: Group Edge Routing	24

15	Topology Diffing	24
16	Temporal Analysis	24
17	Traffic Flow Animation	25
18	Annotation Markup	25
19	Interactive Editor	25
20	Visual Effects	26
21	Graph Analytics	27
22	Filter DSL	27
23	Input Validation	28
24	Quality Metrics	28
24.1	Metric Families	29
24.2	Clarity Score and CI Integration	29
25	Rendering Pipeline	30
25.1	SVG Renderer	30
25.2	Level of Detail	30
25.3	Terminal Renderer	31
25.4	Export Formats	31
26	Pluggable Node Renderer	31
27	API and Usage	32
27.1	Rust API	32
27.2	WASM / JavaScript	32
28	Deployment	32
29	Performance	33
30	Conclusion	34
31	Limitations	34
32	Future Work	35
33	YAML Configuration Examples	35
33.1	Basic Campus Network (§2)	35
33.2	Geographic CDN (§2)	36
33.3	Isometric Multi-Layer (§2)	36
33.4	Parallel Edges and Bond Groups (§2)	36
33.5	Subway Layout (§2)	37
33.6	Pipeline Composition (§4)	37
33.7	Interface Configuration (§9)	37
33.8	Edge Gradients (§10)	37
33.9	Metric Encoding (§11)	37
33.10	Traffic Animation (§17)	37
33.11	Annotations (§18)	38
33.12	Assigning Effects (§20)	38
	Revision History	40
LAG	Link Aggregation Group	5
DSL	Domain-Specific Language	27
LOD	Level of Detail	8

1 Introduction

NetVis is a network topology layout and visualization engine. Given a graph of network devices and connections (from a YAML file, NetBox [1] instance, or LLDP discovery), it produces publication-quality SVG diagrams, interactive HTML visualizations, or PNG/PDF exports. It addresses the gap between general-purpose graph layout tools (Graphviz [2], OGDF [3], D3.js [4]) and manual diagramming applications (yEd [5], Visio).

1.1 Design Philosophy

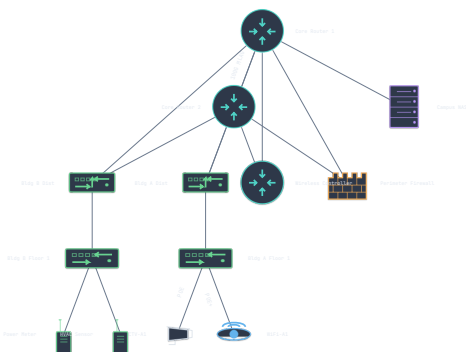
Two principles guide every design decision in NetVis:

Domain-specialized composition. Network topologies have exploitable structural regularity—hierarchical tiers, geographic site placement, protocol-layer stacking. Rather than using a single general-purpose layout algorithm, NetVis decomposes visualization into a pipeline of composable refiners, each specialized for one aspect of the output.

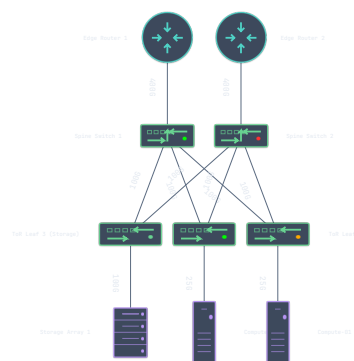
Quality by default. The system should produce readable, labeled, styled diagrams without manual post-processing. Quality metrics (crossings, overlaps, label readability) are measured as part of the rendering pipeline, not as an afterthought.

1.2 Output Gallery

Figures 1 through 3 show representative outputs across different layout modes and topology scales. Every diagram was generated automatically from YAML with no manual adjustment.

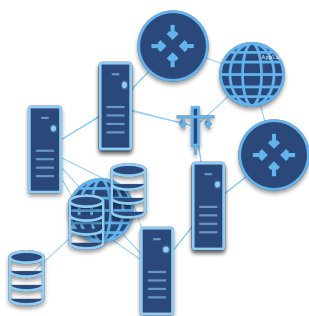


(a) Enterprise campus (hierarchical layout, mixed IoT/Security)

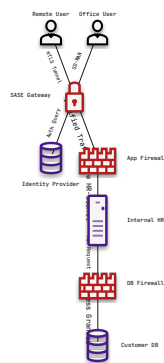


(b) Modern Datacenter (spine-leaf Clos fabric)

Figure 1. Representative NetVis outputs utilizing the multi-part semantic rendering system. All diagrams generated automatically from YAML with no manual adjustment.

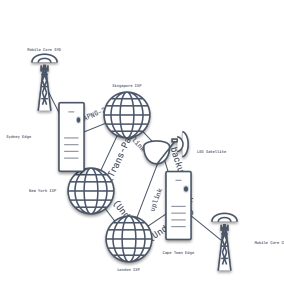


(a) AWS Cloud Architecture (force-directed, blueprint theme)

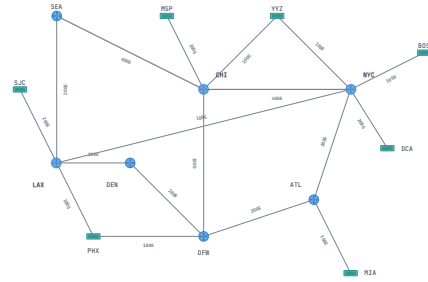


(b) Zero Trust Architecture (high-contrast theme, data-driven LEDs)

Figure 2. Production-grade architectural case studies demonstrating customized themes and semantic icons.



(a) Global ISP Backbone (geographic layout, satellite transit)



(b) Subway layout (octilinear, ring backbone)

Figure 3. Specialized layout modes. Left: Geographic layout utilizing precise lat/lon coordinates. Right: subway layout constrains all edges to 0/45/90 degree routes for transit-map clarity.

1.3 Audience

This document targets three audiences: engineers embedding NetVis as a library in a network management platform (via Rust, Python, or WASM); contributors to the NetVis codebase; and researchers who want to understand the algorithms in detail. Familiarity with basic graph theory is assumed; familiarity with network engineering concepts (spine-leaf, OSPF, BGP) is helpful but not required.

1.4 How to Read This Document

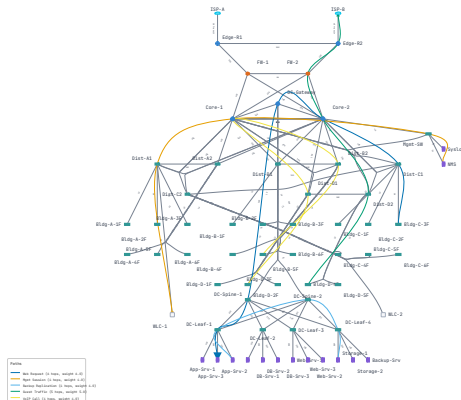
- **Quick start:** Read §2 (By Example), then §27 (API and Usage).
- **Data model and architecture:** §3–§4.
- **Deep dive into algorithms:** §5–§24.
- **Analysis features:** §15–§19 (diff, timeline, traffic, annotations, editor).
- **Deployment and configuration:** §28.
- **Known issues:** §31.

2 NetVis by Example

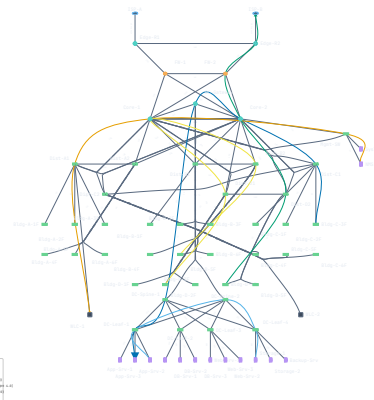
This section walks through representative capabilities using minimal YAML examples.

2.1 Basic Campus Network

The minimal topology YAML requires only name, nodes, and edges. The rendering section is optional; NetVis selects a suitable layout automatically. (See §33 for full YAML snippets).



(a) Enterprise campus, light theme



(b) Same topology, dark theme

Figure 4. The same enterprise campus topology in two themes. The YAML is identical; only `theme:` changes. See §6 for the full theme gallery.

2.2 Layout Algorithm Comparison

The preset or layout field selects the layout engine. Figure 5 shows the same datacenter topology rendered with four different engines.

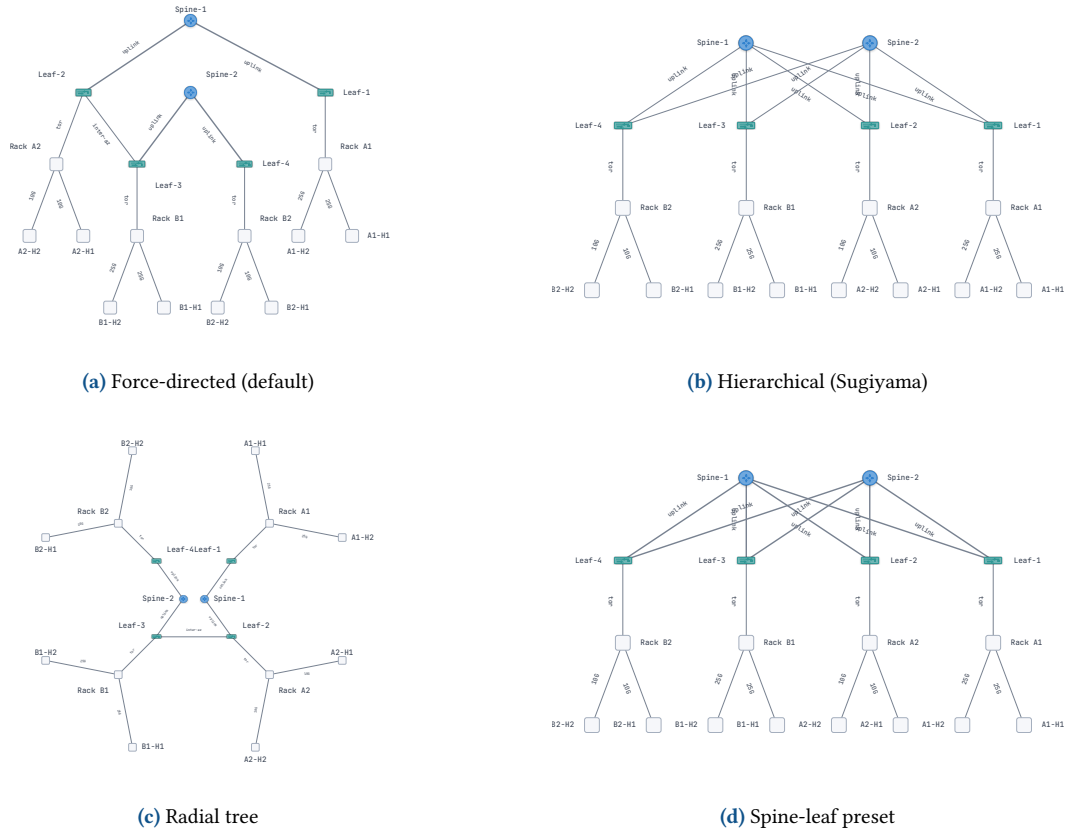


Figure 5. The same 48-node datacenter topology rendered with four layout algorithms. Hierarchical most clearly exposes the two-tier structure; radial is best for hub-centric views; force-directed and spine-leaf are both suitable for ad-hoc exploration.

2.3 Geographic CDN

Nodes with lat/lon attributes use the geographic layout with great-circle arc edges and a background map layer (see §33).

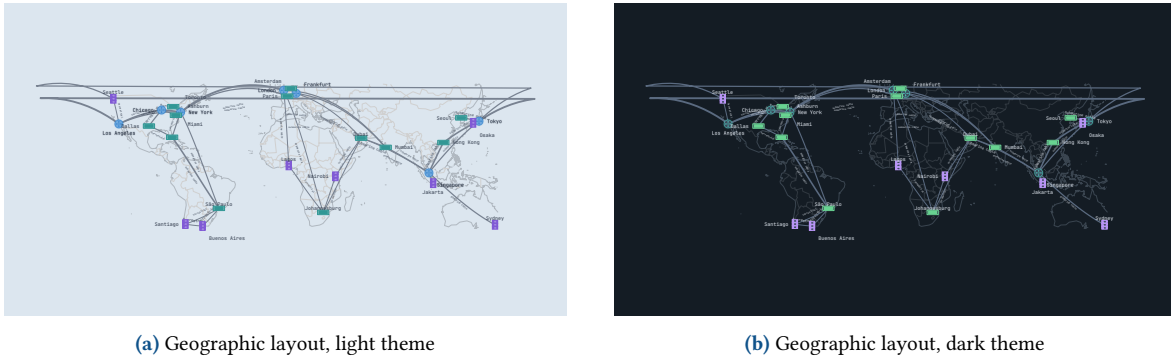


Figure 6. Global CDN in geographic layout. Great-circle arc edges and background map layer. The dark theme is preferred for geographic diagrams as map features remain legible.

2.4 Isometric Multi-Layer

The isometric layout renders protocol stacks or layered architectures with a pseudo-3D perspective. Each layer runs an independent force-directed sub-layout (see §33).

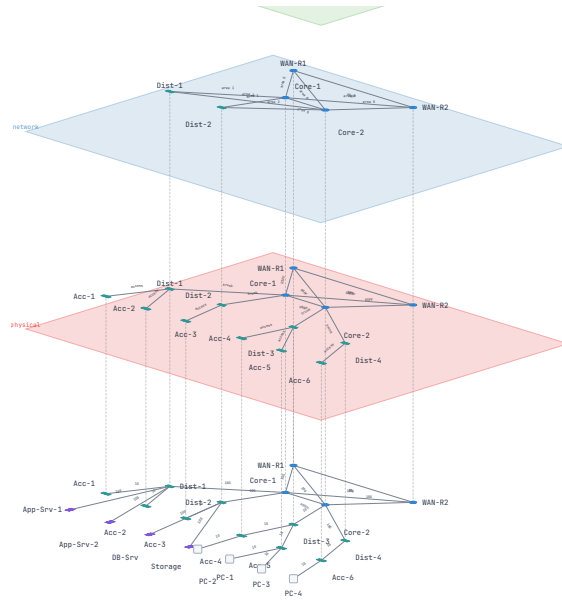


Figure 7. Isometric multi-layer: three protocol planes (Physical, Network, Routing) separated along the Z-axis. Layer planes render as filled SVG polygons. The projection angle is $\phi = 18^\circ$ with 350 scene units of layer spacing.

2.5 Parallel Edges and Bond Groups

Multiple physical links between the same pair of nodes are grouped into an EdgeGroup. The `parallel_mode` field selects the rendering mode; bond groups aggregate bandwidth automatically (see §33).

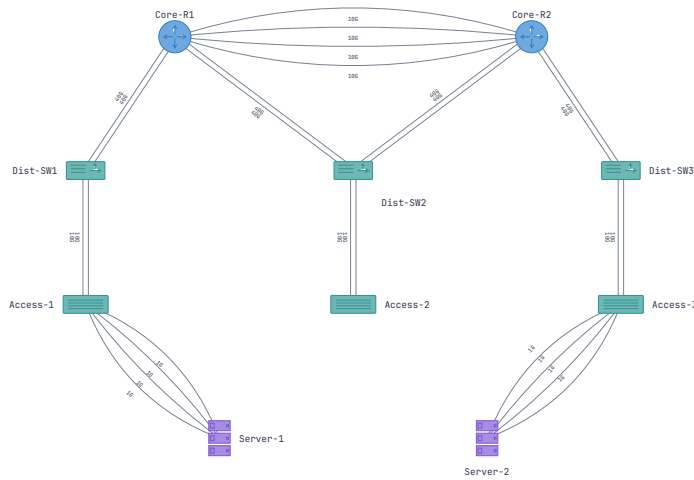
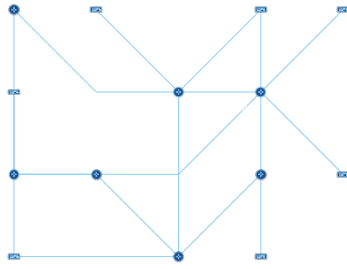


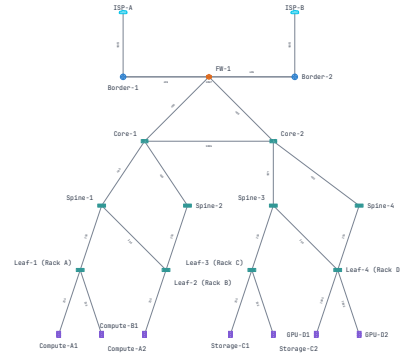
Figure 8. Parallel edge rendering modes on a datacenter topology. The diagram demonstrates Bundled (core Link Aggregation Group (LAG)), Offset (redundant uplinks), BondingIcon (bonded access), and Stacked (server NICs) modes. See §7 for details.

2.6 Subway Layout

The subway layout constrains all edges to 0° , 45° , or 90° routes and snaps nodes to a uniform grid. The blueprint theme complements the geometric precision (see §33).



(a) Subway layout, blueprint theme



(b) Orthogonal layout

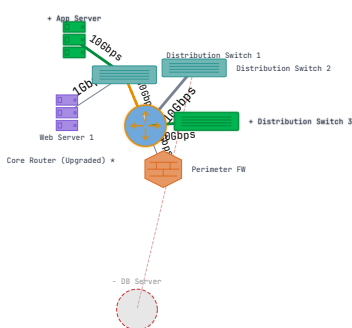
Figure 9. Octilinear and orthogonal layouts. Both enforce geometric discipline: subway at 0/45/90 degrees; orthogonal at strict Manhattan-style horizontal and vertical segments.

2.7 Topology Diff and Analysis

NetVis can structurally compare two topologies and produce a visual diff (added in green, removed in red, modified in amber) and compute graph analytics:

Topology diff and analytics

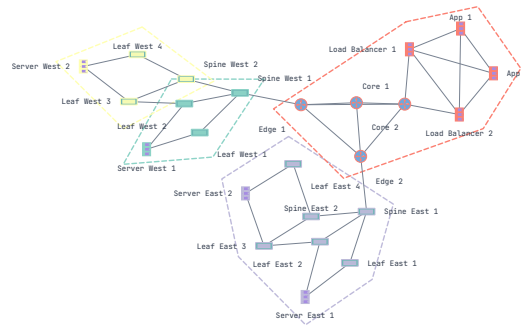
```
# Visual diff between topology versions
netvis diff old.yaml new.yaml -o diff.svg
# Graph analytics overlay
netvis --input topology.yaml --analytics -o analytics.svg
```



(a) Topology diff: green=added, red=removed, amber=modified

Topology Diff

- Added (4)
- Removed (2)
- Modified (2)



(b) Community detection with group boundaries

Figure 10. Analysis features. Left: topology diff visualizes structural changes between two versions. Right: automatic community detection partitions nodes into groups with colored boundary overlays.

3 Data Model

NetVis uses a directed graph as its core data structure, built on the petgraph library [6]. The core type, NetVisGraph, is a StableGraph<NodeData, EdgeData>; the StableGraph variant is used because node and edge indices remain valid after removal—important for the editor’s undo/redo system.

3.1 Node and Edge Data

Field	Type	Purpose
<i>NodeData</i>		
id	String	Unique identifier (e.g., "router-fra-01")
label	Option<String>	Display text
node_type	Option<String>	Device type driving icon and default styling
attributes	HashMap	Arbitrary metadata for metric encoding and filter DSL
position	Option<Point2D>	Layout hint; overrides engine if set
group	Option<GroupId>	Group membership (community detection, composite)
layer	Option<LayerId>	Isometric layer membership
<i>EdgeData</i>		
label	Option<String>	Edge label
edge_type	Option<String>	Semantic type: p2p, trunk, submarine, ...
directed	bool	Default false
bond_group	Option<String>	LAG/bond group identifier
weight	f64	Default 1.0; used as spring constant in force-directed
attributes	HashMap	Arbitrary metadata

Table 1. Core graph data fields.

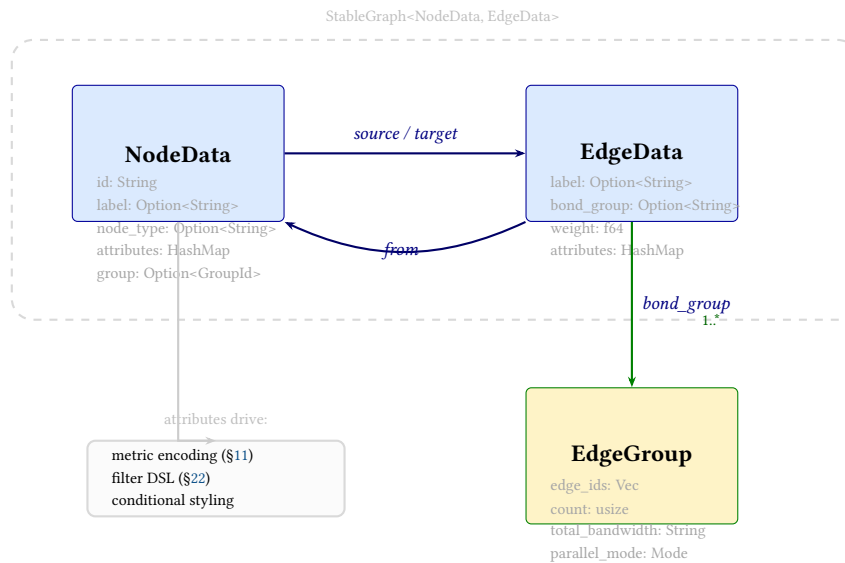


Figure 11. Data model entity relationships. NodeData and EdgeData live in a `petgraph::StableGraph` whose indices remain stable across removals. Edges sharing a `bond_group` are aggregated into an `EdgeGroup` with computed bandwidth totals. The attributes `HashMap` drives metric encoding, filter DSL evaluation, and conditional styling.

3.2 Edge Groups

When multiple edges connect the same pair of nodes, NetVis groups them into an `EdgeGroup` that carries aggregate metadata: the member `edge_ids`, a count, an automatically computed `total_bandwidth` (e.g., four edges labeled "10G" produce "40G"), and a `parallel_mode` (see §7).

Note

The `node_type` field drives device-specific rendering: icon shape, default color, and default size. The full set of built-in node types is described in §8. Unknown values render as circles.

3.3 Scene Graph

Layout engines consume a `NetVisGraph` and produce a `Scene`: a flat list of positioned nodes (with shapes, colors, labels), edges (as Bézier paths), label anchors, group boundaries, and layer planes. Refiners transform the scene between layout and rendering. The scene at 5,000 nodes consumes approximately 9.7 MB of heap.

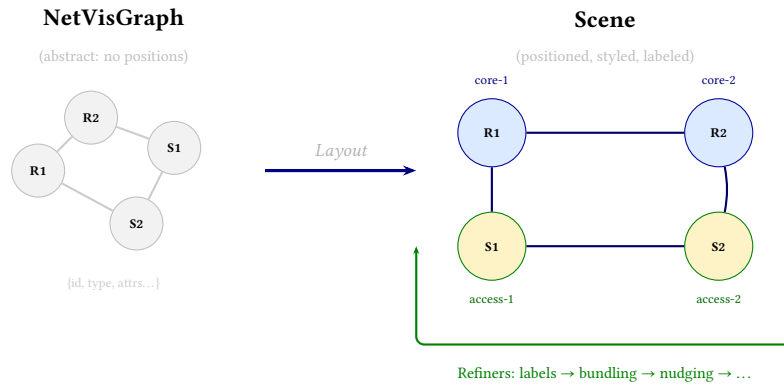


Figure 12. Graph-to-scene transformation. The abstract NetVisGraph (left) carries only topology structure. A layout engine assigns positions, shapes, and colors to produce a Scene (right). Refiners iteratively transform the scene—placing labels, bundling edges, nudging overlaps—before the renderer draws it.

4 Architecture

NetVis is organized as a six-layer pipeline.

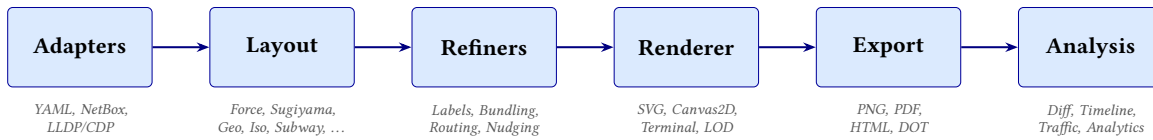


Figure 13. The NetVis six-layer pipeline.

The pipeline stages are:

1. **Adapters.** Stateless converters that ingest topology data from YAML, NetBox [1] REST API, or LLDP/CDP JSON, each implementing `fetch/validate/to_graph`.
2. **Layout engines.** Pluggable algorithms behind a common `Layout` trait. A layout takes a `NetVisGraph` and returns a `Scene`. Nine engines ship: force-directed (Standard and Enterprise variants), hierarchical (Sugiyama [7]), radial tree, geographic (Mercator), isometric multi-layer, orthogonal, subway (octilinear), tree, and composite (multi-region).
3. **Refiners.** Post-layout transformations. Current refiners include label placement (R^* -tree-accelerated), edge bundling (FDEB), constraint solving, overlap removal, edge nudging, label nudging, edge clipping, smart label snapping, starburst grouping, boundary bundling, port docking, and orthogonal edge routing.
4. **Renderers.** SVG (primary), Canvas2D (browser), and ASCII terminal (`netvis show`). All support Level of Detail (**LOD**) suppression and viewport culling.
5. **Export.** SVG is optionally converted to PNG (via `resvg`), PDF (via `svg2pdf`), DOT (Graphviz interoperability), or self-contained interactive HTML.
6. **Analysis.** Post-render modules: topology diffing, temporal snapshots, traffic flow animation, SVG annotation markup, and graph analytics.

4.1 Pipeline Composition

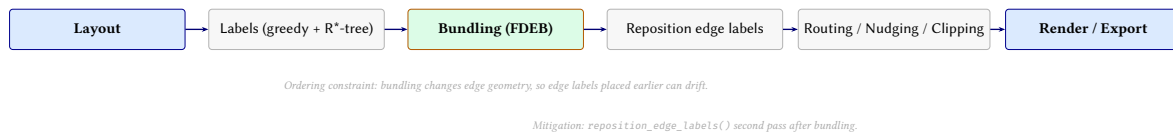


Figure 14. Refiner ordering constraint: edge bundling is geometry-changing, so it is followed by an explicit edge-label reposition pass. This keeps the pipeline composable while preserving legibility.

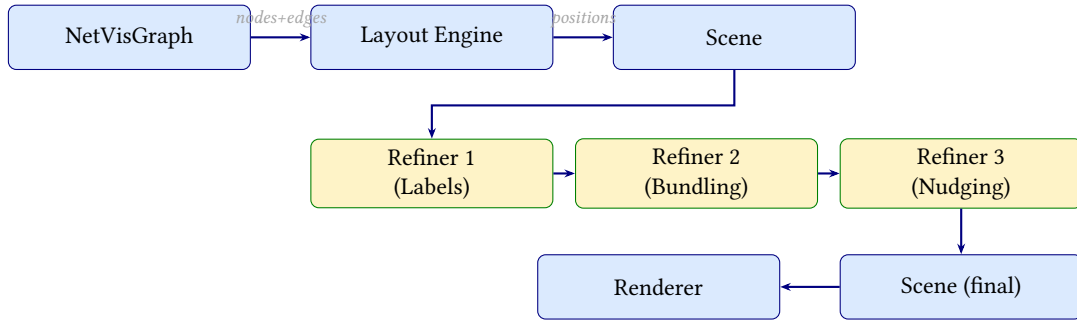


Figure 15. Pipeline composition: layout engine produces a Scene, which flows through a chain of refiners before reaching the renderer. Each refiner implements trait `LayoutRefiner` and may add, remove, or reposition scene elements without knowledge of the other refiners.

In YAML, the refiner chain is expressed explicitly (see §33). The trait boundary prevents cross-component optimization (e.g., bundling cannot influence label placement during the same pass). This is mitigated by `reposition_edge_labels()`, which re-places labels after bundling detects significant curvature changes.

4.2 Scene Graph as Intermediate Representation

The Scene struct carries fully positioned nodes (shapes, colors, labels) and edges (Bézier paths). This separation of concerns means renderers are simple—they draw exactly what the scene contains—at the cost of carrying all rendering data in memory (see §3 for heap measurements).

5 Layout Algorithms

NetVis ships nine layout engines, each implementing the `Layout` trait. The algorithm is selected via the `layout` or `preset` YAML field, or the `--layout/--preset` CLI flags.

5.1 Force-Directed Layout

Position nodes in 2D such that connected nodes are close and unconnected nodes are separated, following the spring-electrical model of Fruchterman and Reingold [8]. NetVis ships two variants and selects between them automatically: the **Standard engine** ($O(n^2)$, wraps the `fjandra` library with Verlet integration [9]) for graphs under 2,000 nodes; the **Enterprise engine** (Barnes-Hut $O(n \log n)$ [10], custom implementation with opening angle $\theta = 0.9$) for larger graphs.

Key parameters: velocity damping $\gamma = 0.6$; alpha decay $1 - 0.001^{1/300}$; iteration count $\min(300, 50 + \lfloor n/10 \rfloor)$; minimum edge length 60 pt (enforced post-simulation).

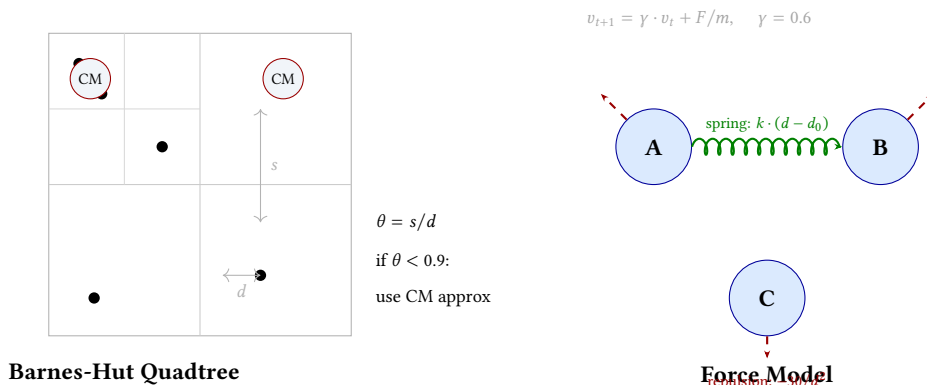


Figure 16. Force-directed layout internals. **Left:** Barnes-Hut quadtree ($O(n \log n)$) approximates distant node groups as a single center-of-mass (CM) when opening angle $\theta = s/d < 0.9$. **Right:** each node experiences attractive spring forces along edges and repulsive charge forces from all other nodes, with velocity damping $\gamma = 0.6$.

5.2 Hierarchical Layout (Sugiyama)

Designed for DAG-structured or tiered networks (e.g., spine-leaf datacenters), the hierarchical layout uses the four-phase Sugiyama framework [7]: (1) cycle removal by edge reversal; (2) longest-path-first layer assignment; (3) crossing minimization via barycenter or median heuristic; (4) coordinate assignment to minimize edge bends. Performance: 19 ms at 1,000 nodes.

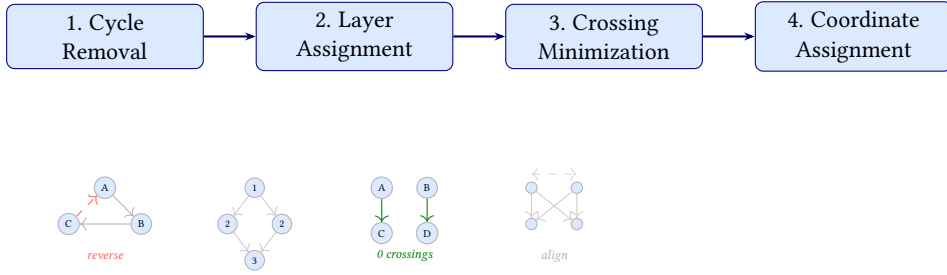


Figure 17. Sugiyama four-phase pipeline. Each phase progressively refines the layout: cycle removal ensures a DAG; layer assignment creates tiers; crossing minimization reorders nodes within layers; coordinate assignment produces final positions.

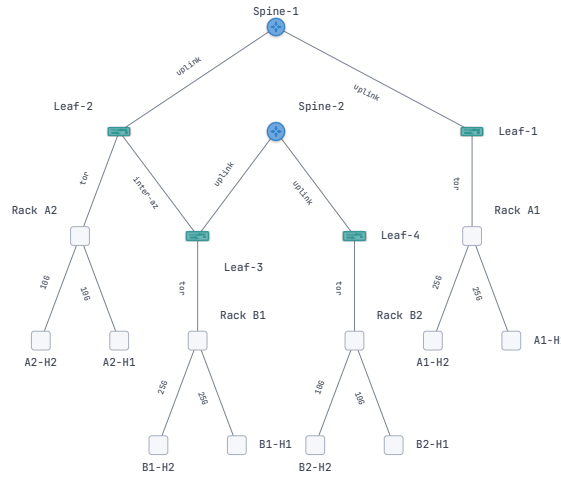


Figure 18. Hierarchical (Sugiyama) layout: a datacenter with clearly separated tiers. The algorithm minimizes edge crossings while preserving the spine → leaf → access hierarchy.

5.3 Geographic Layout

Nodes with lat/lon attributes are projected using an equirectangular projection with dynamic scale normalization $s = 1000 / \max(\Delta x, \Delta y)$. Edges render as geodesic great-circle arcs (Vincenty approximation, 20 segments).

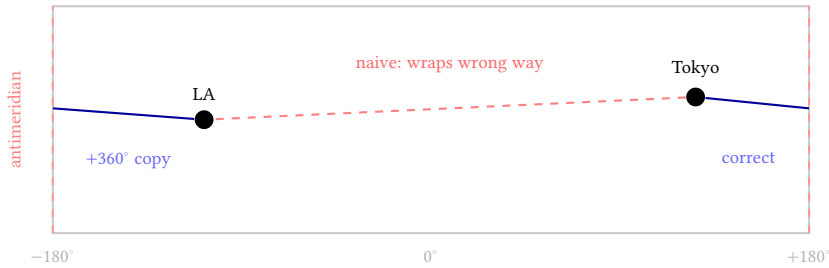


Figure 19. Antimeridian wrapping. An edge from Tokyo to Los Angeles naively crosses the full map width (red dashed). NetVis detects the projected discontinuity (>150 px jump) and renders two segments: one exiting at $+180^\circ$ and a 360° -shifted copy entering at -180° .

5.4 Isometric Multi-Layer Layout

Each layer runs an independent force-directed sub-layout. Results are projected isometrically with rotation angle $\phi = \pi/10$ (18°) and configurable layer spacing (default: 350 scene units). Layer planes render as filled SVG polygons. Edge labels are suppressed when edge length falls below 40 scene units.

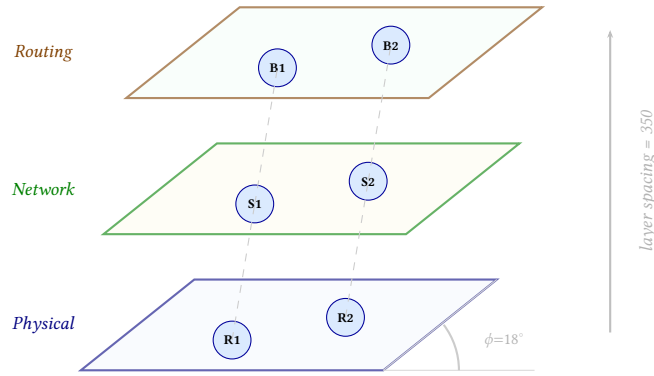


Figure 20. Isometric projection geometry. Each protocol layer runs an independent 2D layout, then all layers are projected with rotation angle $\phi = 18^\circ$. Inter-layer edges (dashed) connect corresponding entities across planes. Layer planes render as filled SVG polygons with configurable opacity.

5.5 Orthogonal Layout

Assigns node positions to a grid and routes all edges as sequences of horizontal and vertical segments with configurable corner radius. A sweep-line algorithm minimizes total edge length while maintaining minimum channel spacing between parallel routes. Suitable for standards-compliant engineering documentation.

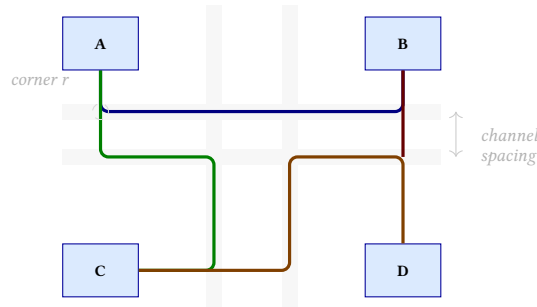


Figure 21. Orthogonal layout with channel routing. Nodes are placed on a grid; all edges are routed as horizontal/vertical segments through dedicated routing channels. A sweep-line algorithm assigns channels to minimize total edge length while enforcing minimum channel spacing. Configurable corner radius rounds the bends.

5.6 Subway Layout

Transit-map style: edges constrained to 0, 45, and 90 degrees; nodes snapped to a regular grid. The engine uses force-directed initialization followed by grid snapping, then routes edges along the octilinear grid with at-most-two-bend routing. Node positions are iteratively adjusted to minimize edge crossings while respecting the grid constraint.

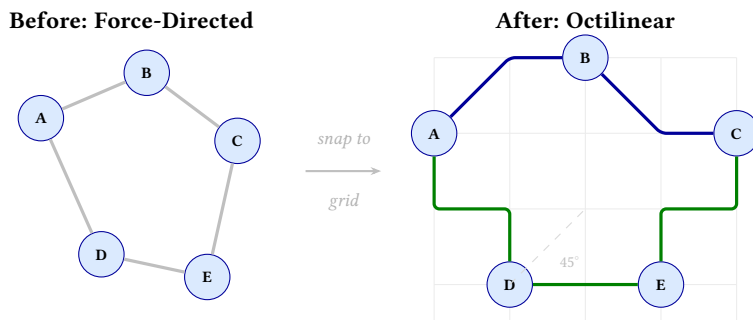


Figure 22. Subway layout: force-directed positions (left) are snapped to an octilinear grid (right). All edges use only 0°, 45°, and 90° segments with at most two bends per edge.

5.7 Radial Tree, Tree, and Composite

Radial tree places a root node at the center with children radiating outward in concentric rings; ring radii scale with subtree size to prevent overlap. **Tree layout** is a simpler algorithm for strict hierarchies: places root at top, arranges children in balanced subtree columns, and skips cycle removal. **Composite layout** assigns different layout algorithms to different sub-graphs within a single canvas, using a parent-level layout to position groups relative to each other.

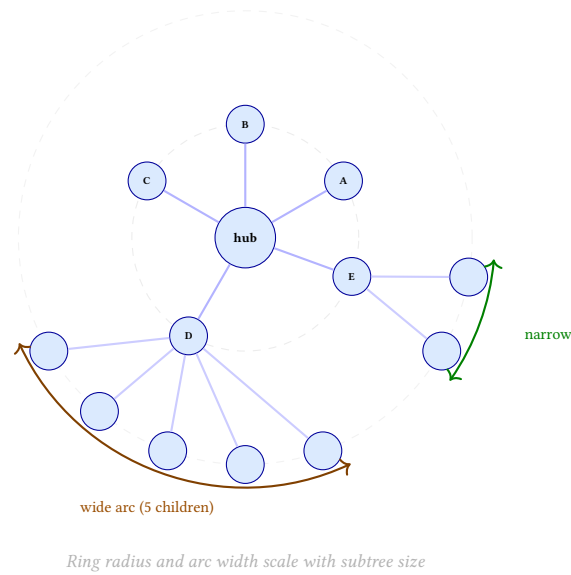


Figure 23. Radial tree layout. The hub sits at center; children radiate outward in concentric rings. Each subtree’s angular allocation is proportional to its descendant count: D’s 5-child subtree gets a wide arc; E’s 2-child subtree gets a narrow arc. This prevents overlap without wasting space.

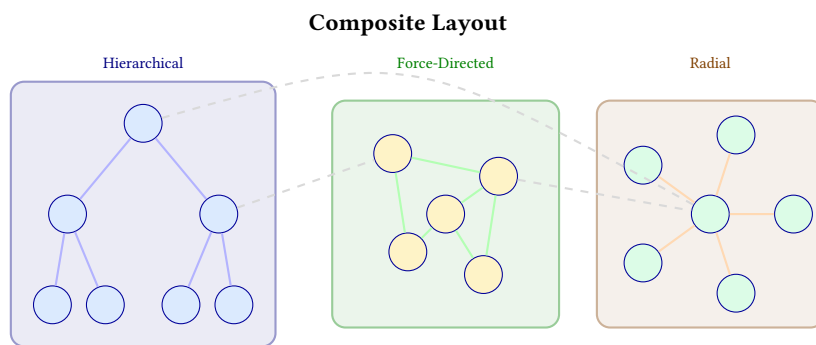


Figure 24. Composite layout: three sub-graphs use different algorithms (hierarchical, force-directed, radial) within a single canvas. A parent-level layout positions the groups relative to each other; inter-group edges (dashed) connect across boundaries.

6 Theme Gallery

NetVis ships seven built-in themes. Each theme defines a complete visual vocabulary: node fill and stroke colors, edge stroke and gradient defaults, label color and font family, background fill, glow and shadow intensity, and group boundary style.

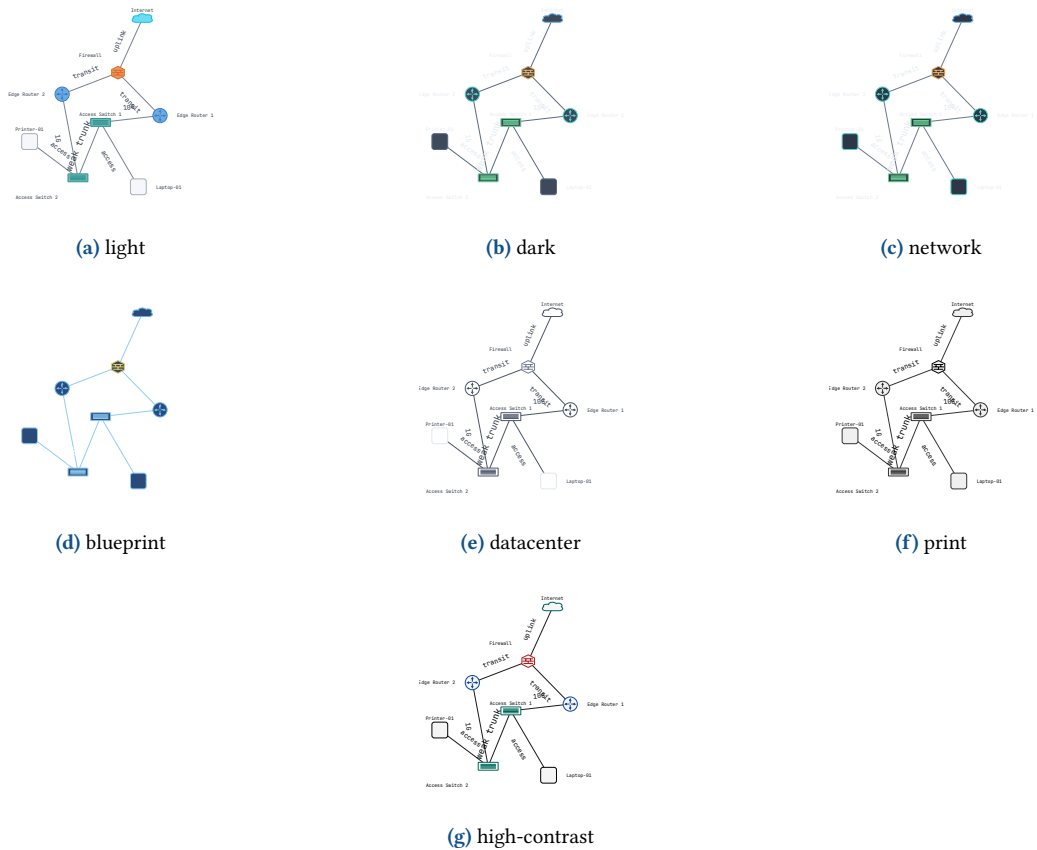


Figure 25. All 7 built-in themes applied to the same 8-node campus topology. Theme is the only YAML difference between these diagrams. The blueprint theme uses a technical-ink style optimized for geometric layouts; high-contrast meets WCAG AA contrast ratios throughout.

7 Parallel Edge Rendering

Network topologies frequently contain multiple physical links between the same pair of devices—redundant uplinks, **LAG** bundles, and NIC-teaming configurations. General-purpose graph tools either render these as overlapping lines (unreadable) or collapse them into a single edge (losing information).

NetVis provides five rendering modes, selectable via `parallel_mode`:

Mode	Behavior
offset (default)	Each link drawn as a visually separate parallel line with perpendicular offset. Four edges produce offsets of $-6, -2, +2, +6$ pixels.
bundled	All links collapsed into one thick line (width scales with $\sqrt{\text{count}}$) with aggregate multiplicity label (e.g., “4x 10G (40G)”).
bonding_icon	Single line with a bracket or chain-link icon at the midpoint.
stacked	Lines drawn very close together (1–2 px offset), creating a thick-line effect without merging strokes.
angled	Curves with different angles/curvatures; distinguishes many parallel links without excessive width.

Table 2. Parallel edge rendering modes.

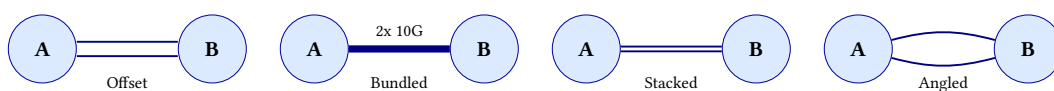


Figure 26. Four parallel edge rendering modes (BondingIcon omitted). Each mode trades visual fidelity for compactness. Offset is clearest for redundant-path diagrams; Bundled is most compact for **LAG** groups.

7.1 Parallel Edge Mode Decision Tree

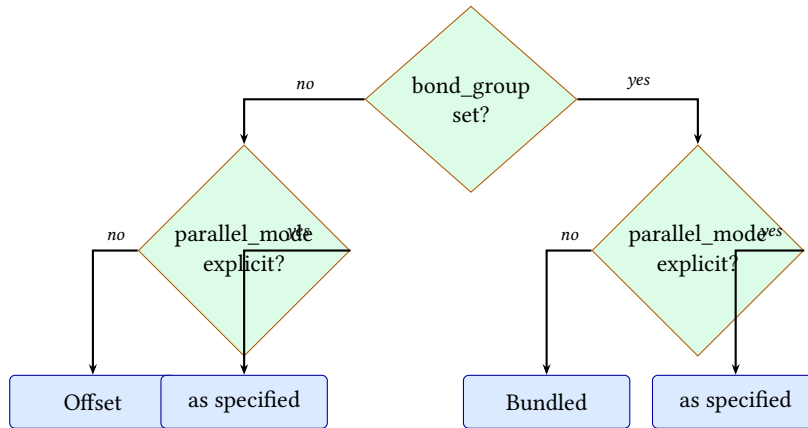


Figure 27. Parallel edge mode selection logic. Bond groups default to Bundled; non-bonded parallel edges default to Offset. An explicit `parallel_mode` always overrides the default.

7.2 Automatic Bandwidth Aggregation

When edges share a `bond_group` identifier, the system automatically parses per-link bandwidth labels and computes aggregate totals using `parse_bandwidth_gbps()`. In Bundled mode, only the total is shown (e.g., “40G”); in other bonded modes, the detail is shown (e.g., “4x 10G (40G)”).

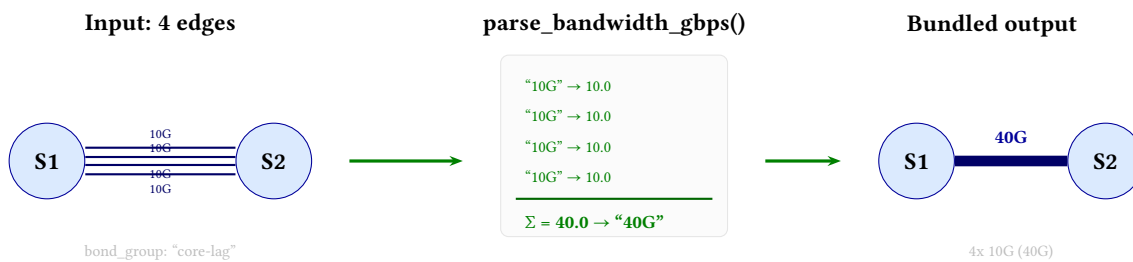


Figure 28. Automatic bandwidth aggregation. Four edges sharing `bond_group`: “core-lag” have their bandwidth labels parsed by `parse_bandwidth_gbps()`, which recognizes SI suffixes (M/G/T). The totals are summed and formatted: Bundled mode shows only “40G”; other modes show “4x 10G (40G)”.

8 Node Shapes and Device Icons

NetVis supports a rich library of geometric shapes and network device icons for node rendering.

8.1 Available Shapes and Icons

Name	Category	Description
circle	Geometric	Default for generic nodes
rectangle	Geometric	Axis-aligned; useful for hosts and servers
ellipse	Geometric	Horizontal ellipse for cloud/WAN nodes
rounded_rectangle	Geometric	Rectangle with configurable corner radius
diamond	Geometric	Rotated square; used for firewalls
triangle	Geometric	Upward-pointing triangle
hexagon	Geometric	Six-sided; useful for AS/BGP speakers

Table 3. Built-in geometric node shapes. Unknown `node_type` values render as circles.

Name	Category	Description
router	Icon	Modern circular icon with inward routing arrows
switch	Icon	Flat rectangular icon with horizontal routing arrows and RJ45 ports
l3_switch	Icon	Switch icon with additional vertical routing arrows
firewall	Icon	Shield or brick-wall icon with offset brick texture
server	Icon	Vertical rack-blade rectangle with drive bay vents
cloud	Icon	Cumulus cloud outline with internal volume lines
load_balancer	Icon	Funnel or balance icon with distribution arrows
database	Icon	Stacked-cylinder icon indicating data tiers
endpoint	Icon	Laptop/workstation or monitor icon
wireless_ap	Icon	Ceiling-mounted access point with signal waves
satellite	Icon	Parabolic dish with receiver arm
mobile_tower	Icon	Lattice cellular tower with fanned antenna arrays
iot_device	Icon	Small sensor enclosure with antenna and vents
vpn	Icon	Padlock symbol representing secure tunnel gateways
internet	Icon	Globe with detailed meridians and latitudes
storage	Icon	Stacked 3D disk drawers
user	Icon	Human silhouette
industrial	Icon	Factory outline with sawtooth roof and smoke
security_camera	Icon	Modern bullet-style CCTV unit with mount

Table 4. Built-in device icons. All icons support multi-part semantic rendering (e.g., dynamic lighting, glowing LEDs).

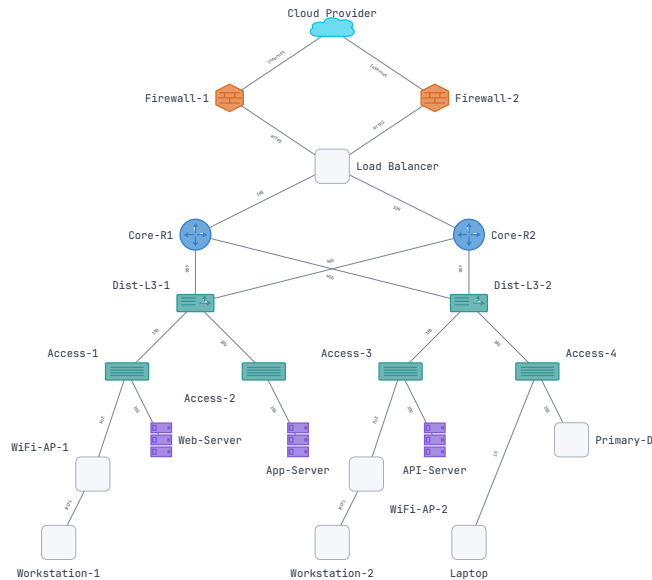


Figure 29. Full device icon library. Each icon shape adapts its size, fill, and stroke to the active theme.

8.2 Multi-Part Semantic Rendering

To bridge the gap between simple generative output and professional, commercial-grade architectural diagrams (e.g., Visio or Lucidchart stencils), NetVis implements a multi-part semantic rendering system for device icons. Rather than rendering a flat SVG shape, each icon is decomposed into semantic layers:

- **Body:** The primary structural outline, utilizing the main theme fill and stroke.
- **BodyTop and BodySide:** For 3D isometric icons (e.g., servers, storage, switches), the rendering engine intercepts these facets and applies simulated lighting. A semi-transparent white overlay (`rgba(255,255,255,0.15)`) is applied to top faces for highlights, while a semi-transparent black

overlay ($\text{rgba}(0,0,0,0.2)$) is applied to side faces for shadows. This true isometric lighting adapts flawlessly to any underlying theme color.

- **Detail:** Internal features such as RJ45 ports, drive bays, or brick patterns are drawn with a subtle, semi-transparent outline (e.g., 60% opacity), ensuring they provide texture without visual clutter.
- **Accent:** Emphasized lines such as routing arrows or signal waves that receive thicker, bolder strokes.
- **Led:** Status indicators mapped directly to node states. The engine parses the `status` or `state` attribute of a node and emits data-driven glowing LEDs (e.g., bright red for critical, green for active, or yellow for warning).

Coupled with injected premium SVG filters—such as `icon-shadow` for a subtle drop shadow granting physical depth, and `icon-glow` utilizing `feGaussianBlur` for light bloom on LEDs—NetVis yields exceptionally polished, data-driven visual components.

9 Interface Rendering

NetVis treats network interfaces as first-class visual objects. Each node can expose its interfaces with configurable visual modes and status indicators.

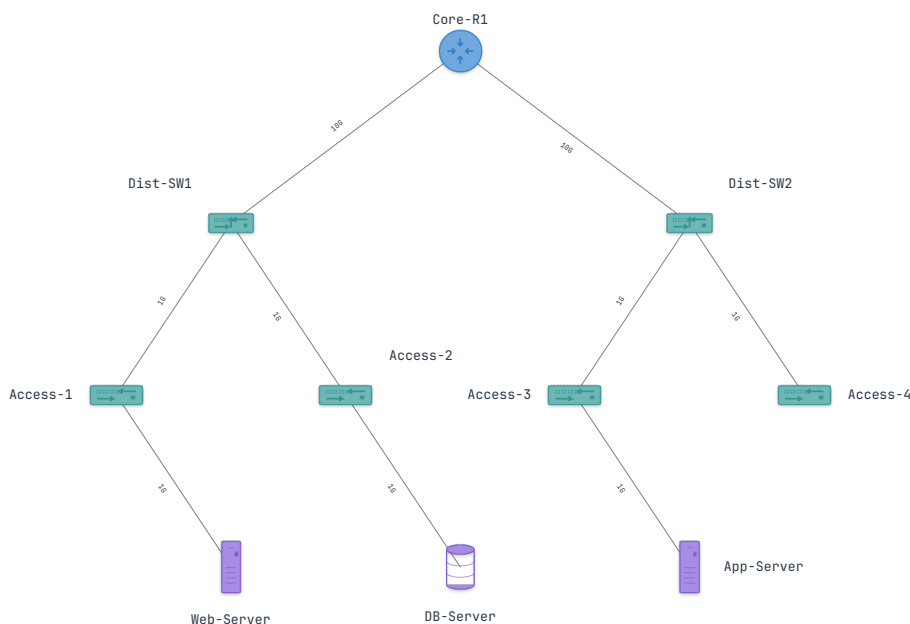


Figure 30. Interface rendering modes: Dot (compact), Port (stub with label), and Detailed (full interface card with status). Status colors are Up (active), Down (gray), AdminDown (strikethrough), and Errors (amber).

Interfaces are configured per-node in YAML (see §33).

The `status` field accepts `up`, `down`, `admin_down`, or `errors`. The `mode` field controls visual detail: `dot` is compact (small circle on node boundary); `port` adds a rectangular stub with label; `detailed` shows the full interface card with speed and operational state. The `marker` field (`none`, `dot`, `square`, `diamond`) controls the endpoint symbol on the edge.

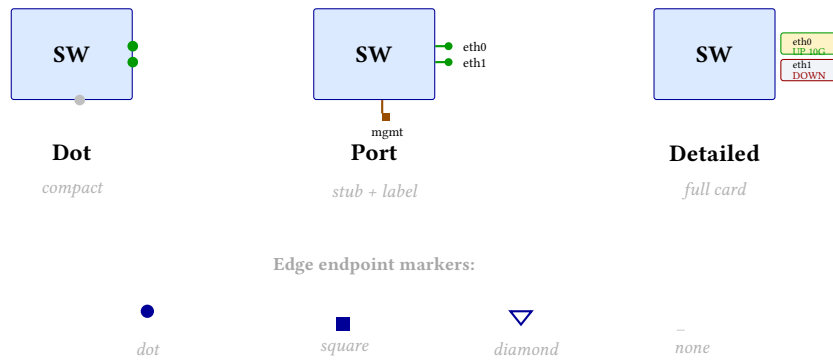


Figure 31. Interface rendering modes. **Dot**: compact circles on the node boundary. **Port**: rectangular stubs with labels. **Detailed**: full interface cards with status and speed. Four marker types control the edge endpoint symbol.

10 Edge Gradients and Path Emphasis

10.1 Edge Gradients

Directional stroke gradients encode flow direction or metric values along an edge. A gradient edge transitions smoothly from a *start color* at the source to an *end color* at the target, implemented as SVG `<linearGradient>` elements aligned to the edge direction:

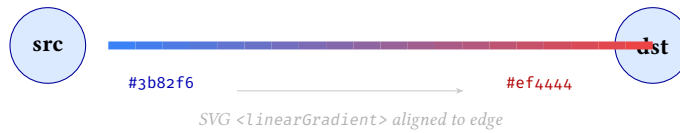


Figure 32. Edge gradient rendering. A directional gradient transitions from *start_color* at the source to *end_color* at the target, implemented as an SVG linear gradient rotated to match the edge angle.

10.2 Path Emphasis

Path emphasis highlights a named subset of edges as a forwarding path. Two emphasis levels (**Primary** for dominant, **Secondary** for backup) combine with three fanout modes (**Fixed**, **Scaled**, **Hybrid**) that control how emphasis interacts with parallel edge groups.

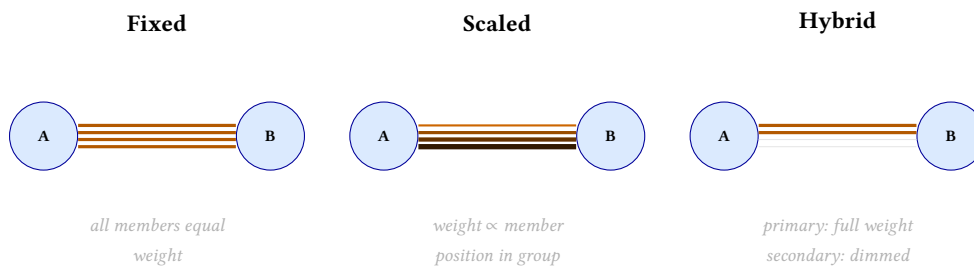


Figure 33. Path emphasis fanout modes on a 4-member LAG. **Fixed**: all members receive equal emphasis weight. **Scaled**: emphasis weight is distributed proportionally across members. **Hybrid**: primary members get full emphasis; secondary members are dimmed.

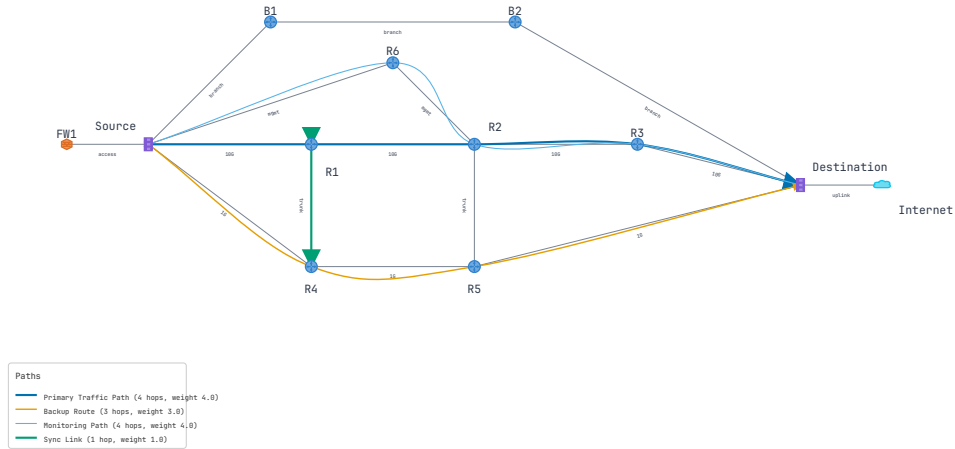


Figure 34. Path emphasis: primary paths in full-weight highlight; secondary in reduced emphasis. Fanout mode *Scaled* is shown on a LAG bundle where emphasis weight scales with member count.

11 Metric Encoding

NetVis maps arbitrary node attributes to visual channels (color, size, opacity), transforming static topology diagrams into live network dashboards. Six sequential palettes are built in: blues, greens, oranges, reds, purples, greys. All are perceptually uniform and work on both light and dark themes.

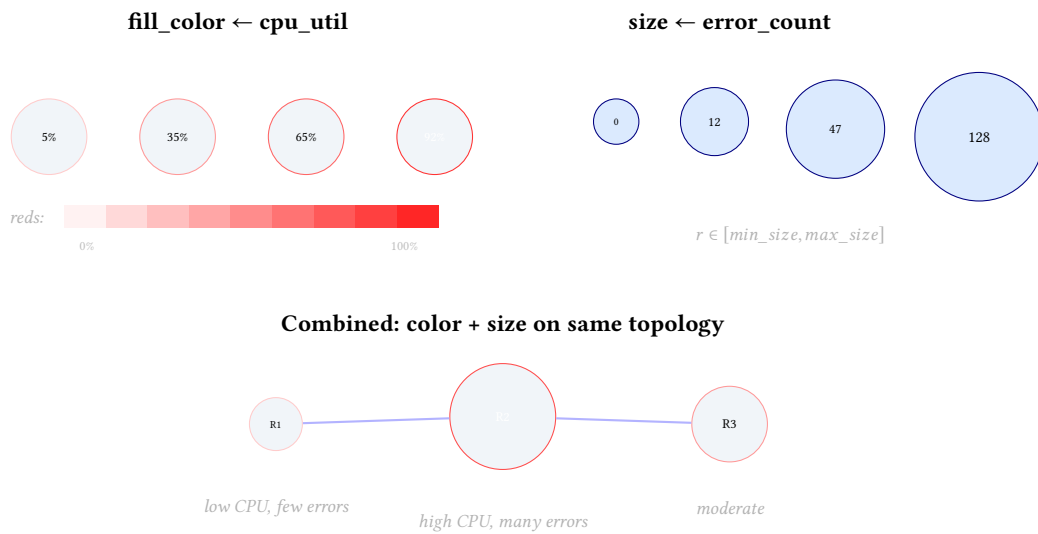


Figure 35. Metric encoding maps data to visual channels. **Top left:** CPU utilization drives node fill color through the “reds” palette. **Top right:** error count drives node radius. **Bottom:** both channels applied simultaneously—at a glance, R2 is the hotspot (large, dark red) while R1 is healthy (small, light).

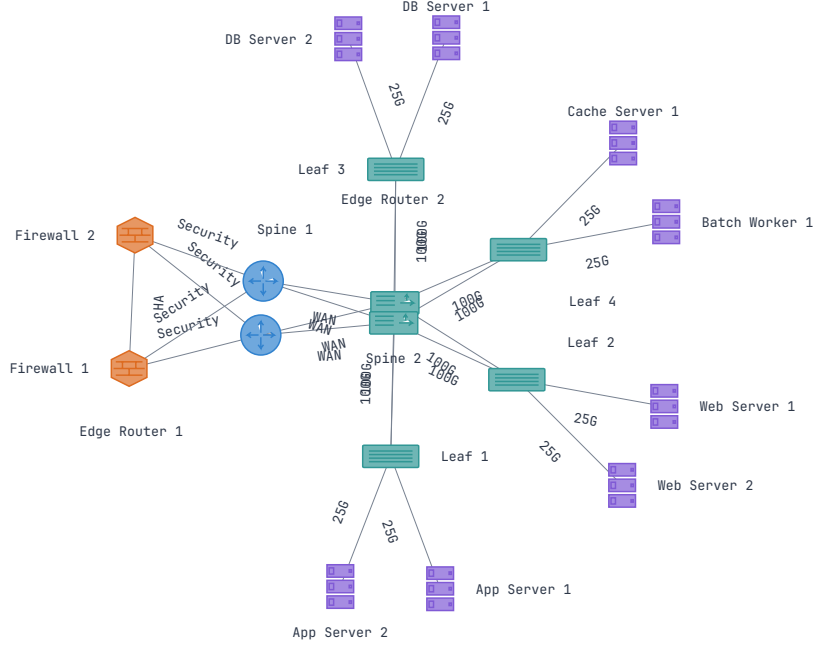


Figure 36. Metric encoding: node fill encodes CPU utilization (reds); node size encodes interface error count; edge stroke encodes link utilization (oranges).

12 Label Placement

12.1 Problem and Algorithm Choice

Placing text labels near their owner without overlapping other labels, nodes, or edges is a variant of the NP-hard point-feature label placement (PFLP) problem [11]. Three approaches were considered: (a) no collision avoidance (Graphviz approach); (b) $O(n^2)$ greedy; (c) R^* -tree-accelerated greedy with $O(\log n)$ collision queries.

NetVis uses option (c). The R^* -tree [12] build cost (~ 5 ms) is recouped after approximately 20 collision queries, which occurs by ~ 50 labels.

12.2 Two-Phase Greedy Algorithm

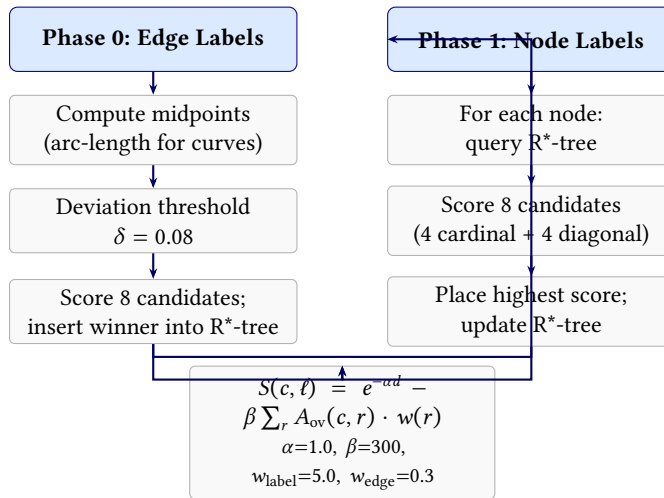


Figure 37. Label placement algorithm. Phase 0 places edge labels first (more constrained by midpoint proximity). Phase 1 places node labels around already-occupied regions. The scoring function penalizes both distance from the owner and overlap with existing geometry.

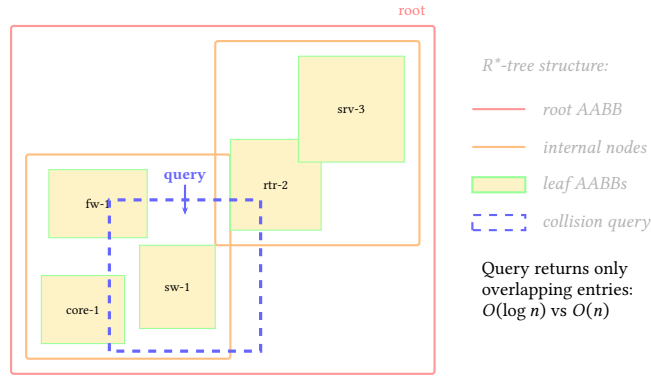


Figure 38. R*-tree spatial index for label collision detection. Occupied regions (node shapes, existing labels, edge exclusion zones) are inserted as axis-aligned bounding boxes. A candidate label position is tested by querying the tree for intersections— $O(\log n)$ per query vs. $O(n)$ brute force.

Edge exclusion zones: Curved/bundled edges can produce 47,000+ segment ABBs. Per-segment exclusion zone registration is skipped for non-straight edges; only a single bounding-box exclusion is registered per curved path.

12.3 Candidate Scoring Visualization

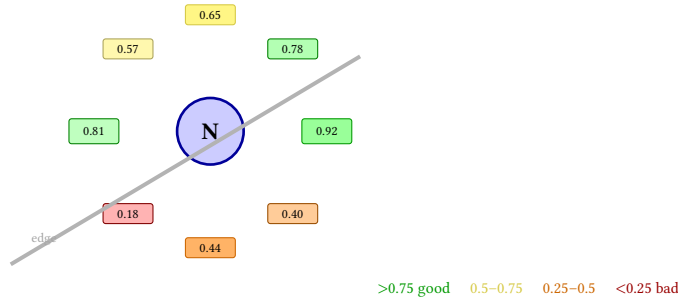


Figure 39. Scoring function applied to 8 candidate label positions around node N. Position scores (0–1) are color-coded from green (good) to red (poor). The right position wins (0.92); the lower-left loses because a diagonal edge passes through it (score 0.18).

12.4 Performance

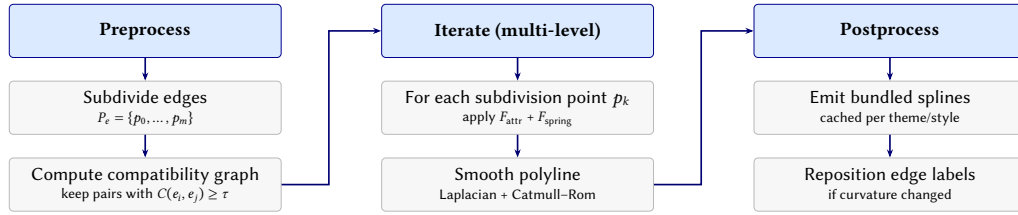
Nodes	Time	Overlaps	Avg. Distance
100	<1 ms	0	8.2 pt
500	12 ms	0	9.1 pt
1000	48 ms	2	10.3 pt
5000	210 ms	18	12.7 pt

Table 5. Label placement performance on spine-leaf topologies (R*-tree acceleration via rstar [13]).

13 Edge Bundling

Dense graphs produce visual clutter from crossing edges. Edge bundling groups spatially compatible edges into bundles, reducing clutter.

13.1 FDEB Force Model



Key idea: move only subdivision points; endpoints stay fixed. Multi-level schedule: increase m and decrease step size across iterations. Bundling happens after initial label placement; edge labels may need a second pass.

Figure 40. Bundling as a pipeline: compatibility graph construction gates which edges interact; the iterative solver updates only subdivision points; smoothing converts a polyline into a visually stable spline.

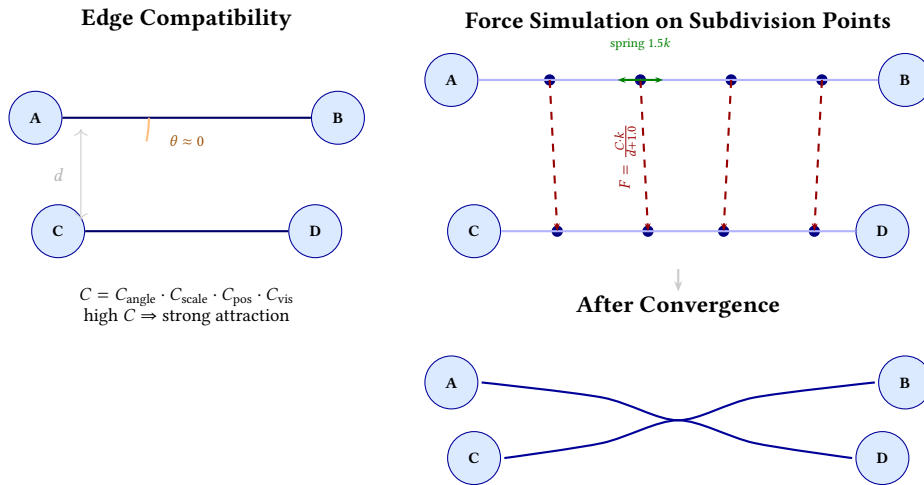


Figure 41. FDEB force-directed edge bundling. **Top left:** edge compatibility scoring—nearly parallel, co-located edges receive high compatibility C . **Top right:** subdivision points along each edge are subject to attraction forces (red, $F = Ck/(d+1)$) toward corresponding points on compatible edges, and spring forces (green, $1.5k$) that preserve edge shape. **Bottom:** after convergence, compatible edges merge into a shared bundle through the midpoint, fanning out to their respective endpoints.

NetVis implements a variant of Holten and Van Wijk’s FDEB [14] with three key modifications: (1) singularity guard $f = k_c/(d + 1.0)$ prevents hairball artifacts; (2) spring coefficient $1.5k$ maintains edge shape; (3) Catmull-Rom tension 0.4 plus 3–5 Laplacian smoothing passes ($\alpha = 0.5$) eliminates high-frequency zigzag artifacts.

Note

Bundling runs *after* label placement. A `reposition_edge_labels()` pass re-replaces labels on edges whose curvature changed significantly post-bundling.

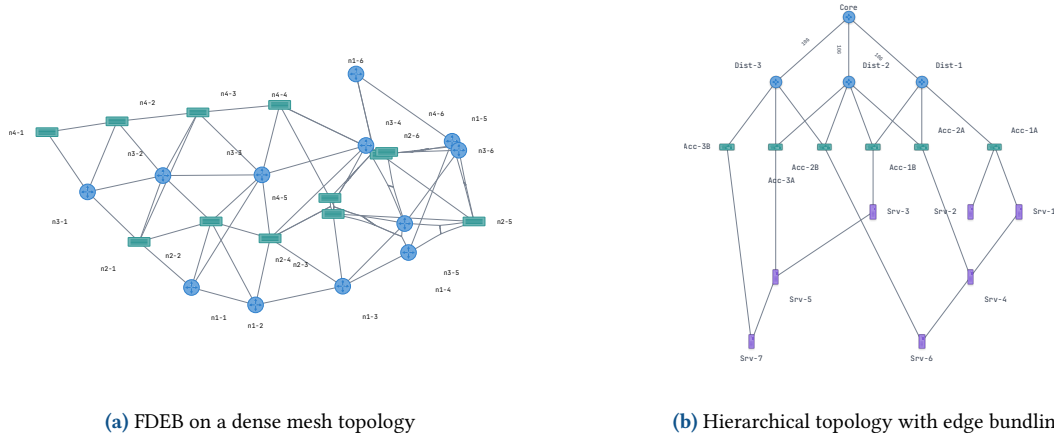


Figure 42. Edge bundling modes. Left: FDEB groups geometrically compatible edges. Right: hierarchical layout combined with edge bundling clarifies cross-level redundancy and server meshes.

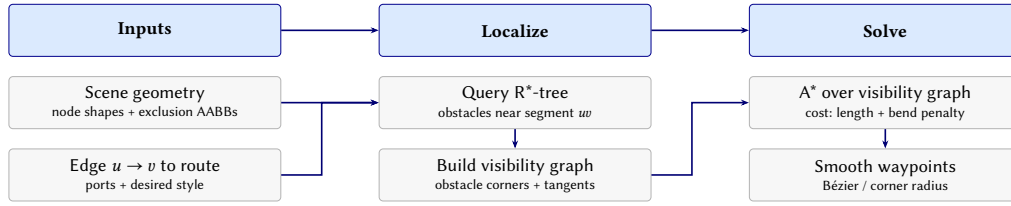
14 Edge Routing and Post-Layout Refiners

NetVis provides ten post-layout refiners, each implementing the `LayoutRefiner` trait. Refiners are composable and can be chained in any order (see §4 and Figure 15).

Refiner	Category	Description
SmartRouting	Routing	Bends edges around obstacles using R*-tree visibility graph + A* pathfinding; waypoints smoothed to Bézier splines (see Figure 44)
OrthogonalRouting	Routing	Converts edges to axis-aligned Manhattan paths with configurable corner radius
PortDocking	Routing	Adjusts endpoints to dock at specific interface positions, distributing connections evenly around the perimeter
BoundaryBundling	Routing	Routes edges through a shared crossing point at group boundaries, fanning out inside
Deoverlap	Spacing	Pushes overlapping nodes apart using repulsive spring force; configurable margin (default: 10 units)
EdgeClipping	Spacing	Clips edge endpoints to the exact node shape boundary for correct arrowhead placement
Nudging	Spacing	Nudges overlapping or near-overlapping edges apart; used after parallel edge grouping
SmartLabel	Labels	8-position snapping; second-pass improvement over initial greedy placement
LabelNudging	Labels	Iterative repulsive force between overlapping label bounding boxes; converges in 10–30 iterations
Starburst	Groups	Positions nodes radially within containment groups; best for AS-level diagrams

Table 6. Post-layout refiners.

14.1 Smart Routing: Visibility Graph



R-tree keeps per-edge work local; routing doesn't scan the whole visibility graph is sparse: only corners that can "see" each other become candidates for rendering concern: keep the path feasible, then make it legible.*

Figure 43. SmartRoutingRefiner as a three-step algorithm: (1) local obstacle selection via an R*-tree query, (2) visibility-graph construction over obstacle features, (3) shortest-path search (A*) followed by spline smoothing. This separates correctness (obstacle avoidance) from aesthetics (curve shape).

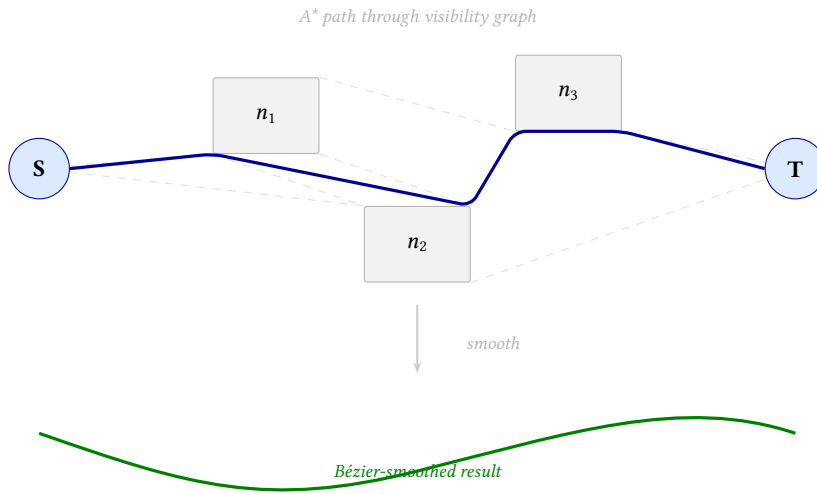


Figure 44. SmartRoutingRefiner. An R*-tree query identifies obstacles near each edge. A visibility graph connects obstacle corners; A* finds the shortest obstacle-free path. Waypoints are smoothed to Bézier splines for natural-looking curves.

14.2 Edge Clipping: Before and After

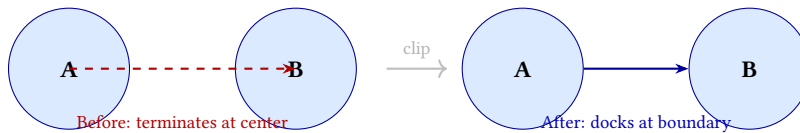


Figure 45. EdgeClippingRefiner: without clipping, edge paths terminate at node centers; with clipping, endpoints dock at the exact node shape boundary, giving correct arrowhead placement regardless of shape.

14.3 Deoverlap: Before and After

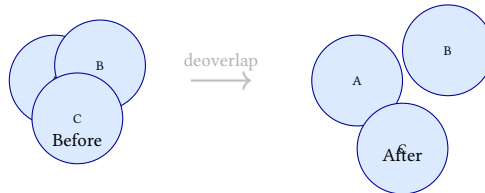


Figure 46. DeoverlapRefiner: applies a repulsive spring force between overlapping node bounding boxes, pushing them apart until the configured minimum margin is satisfied.

14.4 Boundary Bundling: Group Edge Routing

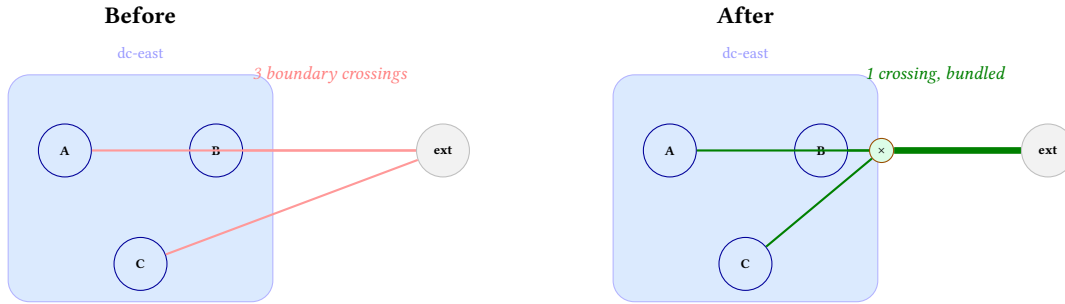


Figure 47. BoundaryBundlingRefiner. **Before:** three edges from internal nodes to an external node each cross the group boundary independently. **After:** all edges fan into a shared crossing point on the boundary and exit as a single bundled edge, reducing visual clutter at group borders.

15 Topology Diffing

The `diff` module computes added, removed, and modified nodes and edges between two `NetVisGraph` instances (by ID and attribute comparison). The `apply_diff_styles()` function color-codes the rendered scene: green for added, red/dimmed for removed, amber for modified. A diff legend is automatically inserted.

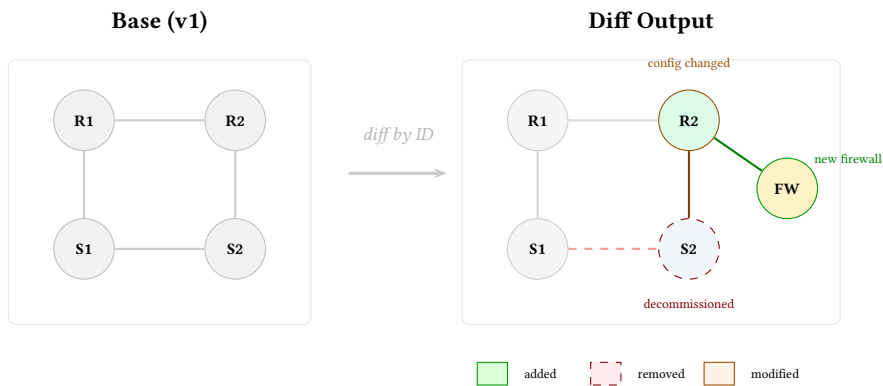


Figure 48. Topology diffing by example. The base topology (v1) is compared to a new version by matching node and edge IDs. The diff output color-codes changes: S2 was decommissioned (red/dashed), R2's config changed (amber), and a new firewall was added (green). Unchanged elements remain gray.

CLI topology diff

```
netvis diff old.yaml new.yaml -o diff.svg
```

16 Temporal Analysis

The temporal analysis system tracks topology evolution over time by scanning a directory for YAML topology files, extracting timestamps from filenames or metadata, and ordering them chronologically. Per-entity lifecycle tracking traces each node's and edge's first appearance, all attribute changes, and removal across snapshots.

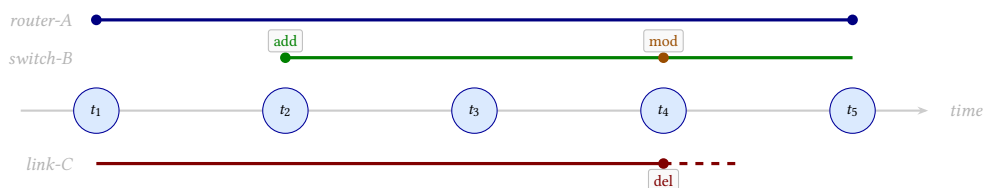


Figure 49. Temporal analysis: entity lifecycle tracking across topology snapshots. Each snapshot (t_1 – t_5) is a YAML file. The timeline module traces first appearance (add), attribute changes (mod), and removal (del) for every node and edge.

Timeline CLI

```
netvis timeline --dir snapshots/  
netvis query --dir snapshots/ --node router-fra-01
```

17 Traffic Flow Animation

The traffic module injects animated flow overlays into rendered SVGs via CSS @keyframes. Each edge can carry utilization (0.0–1.0), direction (forward/reverse/bidirectional), and animation style (dot/dash). Utilization maps to color: green (low) → amber → red (congested). Zero-traffic edges receive CSS dimming (opacity: 0.3). Play/pause and speed controls are injected as an SVG overlay.

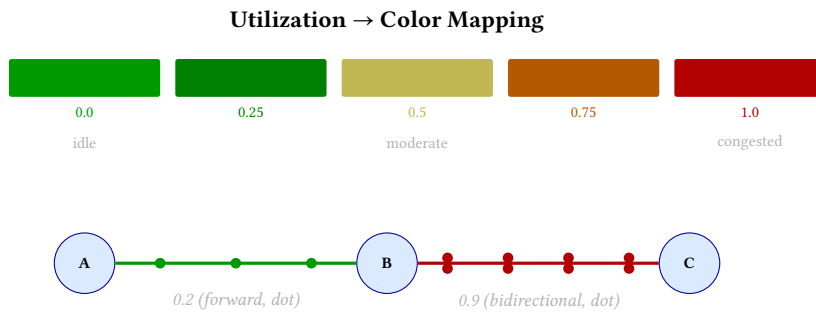


Figure 50. Traffic flow animation. Edge utilization (0.0–1.0) maps to a green–amber–red color ramp. Animated dots travel along the edge path; bidirectional mode shows two opposing dot streams. CSS @keyframes drives the animation in SVG output.

18 Annotation Markup

The annotation module injects text callouts into rendered SVGs, with pointer lines from callout to anchor node. Annotations support layer filtering (show_only_layers, hidden_layers) and three shapes: circle, box, arrow.

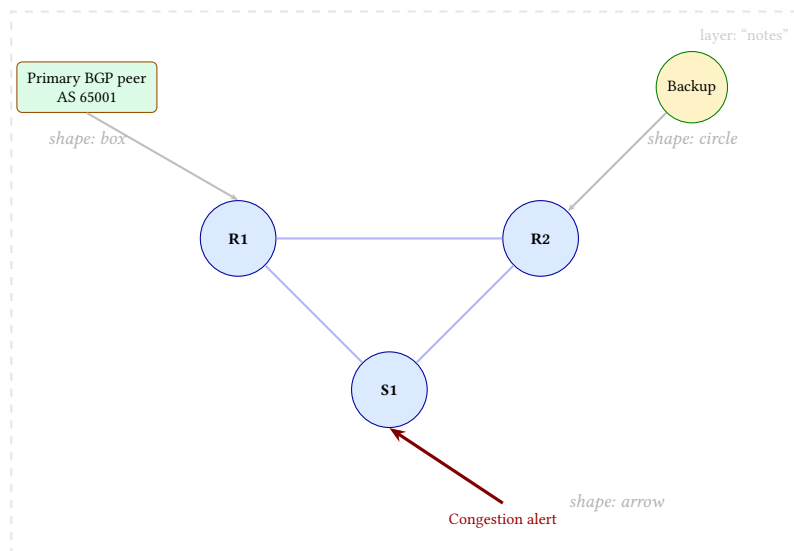


Figure 51. Annotation callout shapes. Each annotation anchors to a node with a pointer line. Three shapes are supported: **box** (rectangular, for multi-line text), **circle** (compact labels), and **arrow** (directional emphasis). Annotations are organized into named layers for selective visibility.

19 Interactive Editor

The editor module provides interactive topology editing for browser deployments via the WASM target. It implements the command pattern: MoveNodeCommand, TransactionCommand, etc., storing only deltas (24 bytes for a node move) rather than full snapshots. An UndoStack with 50-action capacity provides undo/redo, using ~400,000× less memory than snapshot-based undo at 5,000 nodes.

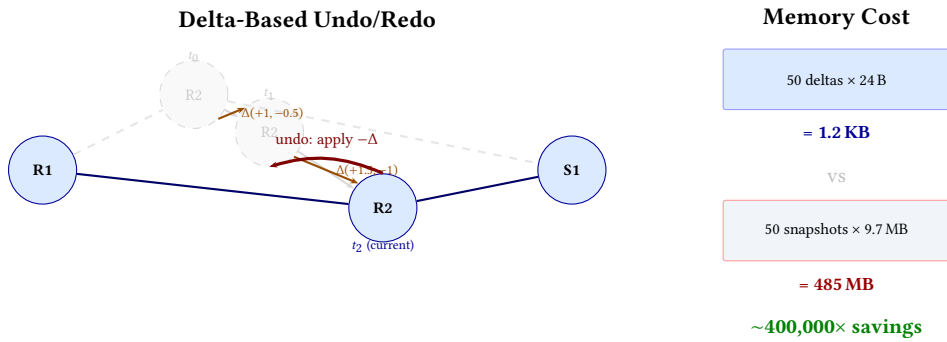


Figure 52. Interactive editor with spatial undo. Node R2 is dragged through three positions; each move stores only a delta ($\Delta x, \Delta y$, 24 bytes) rather than a full scene snapshot. Undo applies $-\Delta$ to restore the previous position. At 5,000 nodes the delta approach uses 1.2 KB vs. 485 MB for snapshot-based undo.

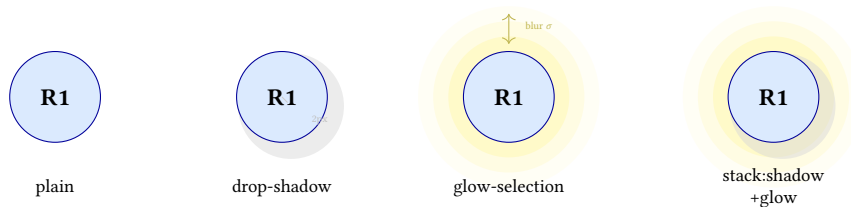
A Lit-based <netvis-editor> Web Component wraps the WASM editor for easy browser integration.

20 Visual Effects

NetVis provides a pluggable visual effects system mapping semantic effect names to SVG filter definitions. The EffectRegistry deduplicates identical filter definitions via content-addressed hashing. Effects are assigned via the highlight field (see §33):

Name	Type	Description
drop-shadow-subtle	Drop shadow	Light offset shadow
drop-shadow-emphasis	Drop shadow	Strong offset shadow
inner-shadow-boundary	Inner shadow	Inset shadow for group boundaries
glow-selection	Glow	Selection highlight
glow-status	Glow	Status indicator
glow-emphasis	Glow	Emphasis marker
stack:<a>+	Composed	Combine any two base effects

Table 7. Built-in visual effects.



SVG <filter>: SourceGraphic $\rightarrow$ feGaussianBlur $\rightarrow$ feOffset $\rightarrow$ feMerge (identical chains deduplicated by content hash)

Figure 53. Visual effects applied to the same node. **Plain:** no filter. **Drop shadow:** offset blurred copy behind the node. **Glow:** colored Gaussian blur halo radiating outward. **Composed:** stack: combines both via SVG feMerge. The EffectRegistry deduplicates identical filter chains by content hash.

All effects adapt to the active theme. Glow intensity and shadow direction vary by theme family.

Effect Budget

SVG filter performance degrades at ~50+ filtered elements. `effects.max_filtered_elements` (default: 50) sets a budget.

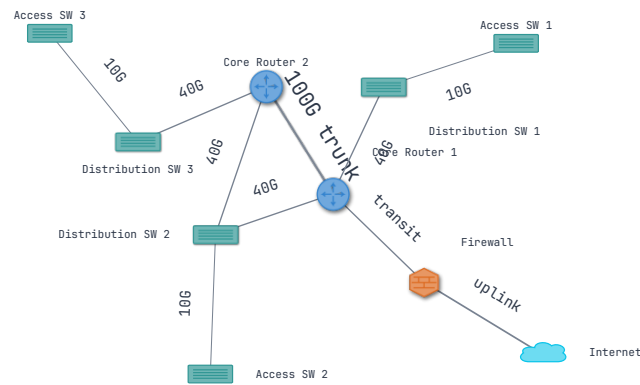


Figure 54. Visual effects showcase. Left to right: drop shadow, glow selection, emphasis glow, and composed stack: drop-shadow+glow. All effects adapt their intensity and color to the active theme.

21 Graph Analytics

The analytics module computes structural graph metrics and injects them as a visual “Network Insights” card. Three metrics are computed: **betweenness centrality** (Brandes’ algorithm, $O(nm)$) identifies critical transit nodes; **graph diameter** (longest shortest path) highlights worst-case latency; **clustering coefficient** measures local redundancy.

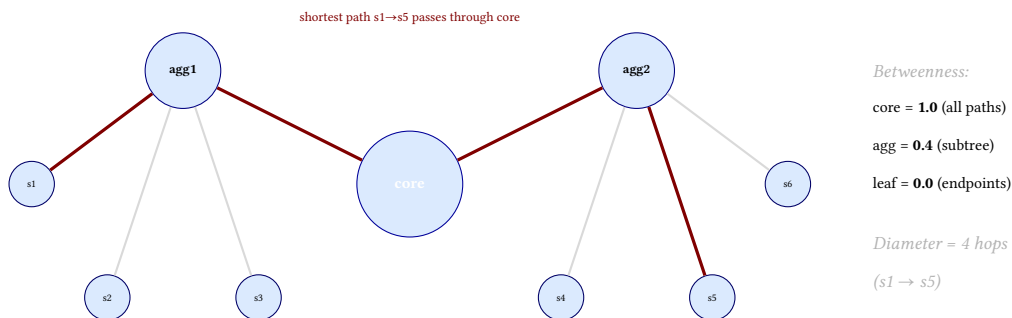


Figure 55. Graph analytics: betweenness centrality visualized. Node size encodes centrality—the core router sits on every shortest path between the two subtrees (centrality 1.0), making it the critical transit point. Aggregation nodes score 0.4 (bridging their subtrees); leaf nodes score 0.0. Graph diameter (longest shortest path) is 4 hops.

Analytics CLI

```
netvis --input topology.yaml --analytics -o analytics.svg
```

22 Filter DSL

NetVis includes a Domain-Specific Language (DSL) for filtering topology elements before rendering. The system comprises three layers: a **parser** that converts filter expressions into an AST (supporting boolean operators, attribute selectors, glob patterns, and set operations); an **evaluator** that walks the AST per node/edge; and an optional **LLM layer** (feature-gated) that translates English queries into filter AST nodes.

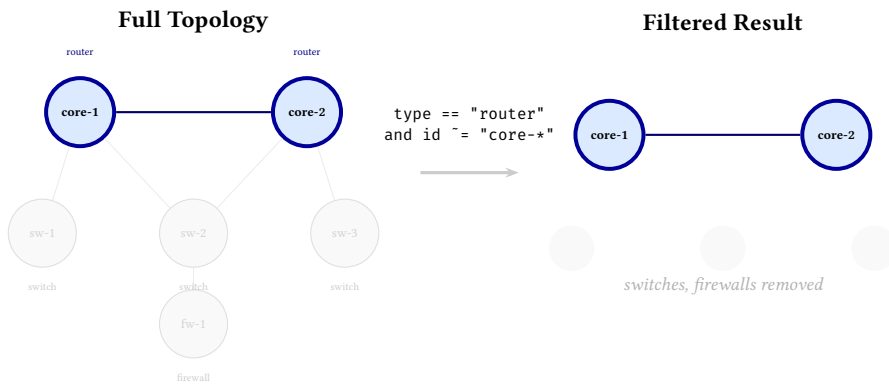


Figure 56. Filter DSL in action. The expression `type == "router" and id ~= "core-*"` matches only the two core routers (highlighted). All non-matching nodes and their incident edges are removed from the rendered output. The filter supports boolean operators, attribute comparisons, and glob patterns.

Filter DSL examples

```
netvis --input t.yaml --filter 'type == "router"'
netvis --input t.yaml \
  --filter 'group == "dc-east" and id ~= "core-*"
```

23 Input Validation

Topology and configuration files are validated against a JSON Schema (generated from Rust types via `schemars`). The `validation::fuzzy` module provides “Did you mean...?” suggestions for misspelled YAML field names using Levenshtein distance (`strsim` crate). Structured diagnostics are returned as `ValidationDiagnostic` entries with severity, location, and suggested corrections.

Schema export and validation

```
netvis schema --type topology > topology-schema.json
netvis validate --input topology.yaml
```

24 Quality Metrics

NetVis computes quality metrics as part of every render, following the aesthetic metrics framework of Purchase [15] and Dunne et al. [16].

24.1 Metric Families

Metric	Description
Edge crossings	R*-tree segment intersection, weighted by angle
Overlap area	AABB intersection between all label/node pairs; sub-pixel (<2 px) ignored
Label readability	Composite of owner distance, font-size variance, reading order
Angular resolution	Minimum angle between consecutive incident edges
Node alignment	Degree to which nodes align on axes or grid lines
Edge clearance	Minimum distance from non-incident edges to nodes
Coincident edges	Detection of duplicate or overlapping edge paths
Contrast	Label color contrast against background (WCAG-derived)
Edge-length variance	Coefficient of variation across edge lengths
Spatial distribution	Evenness of node distribution (entropy-based)
Path straightness	Curvature penalty for bent edges
Viewport coverage	Elements outside viewport bounds
Node-edge overlap	Non-incident edge segments crossing node bounding boxes
Density heatmap	Spatial density for identifying visual hotspots

Table 8. 14 of the 16 quality metric families computed per render. All are aggregated into a unified ClarityReport (0–100).

24.2 Clarity Score and CI Integration

The composite *Clarity Score* (0–100) combines four equally-weighted factors: edge crossings, angular resolution, node spacing, and label overlap. It is designed as a single-number summary for CI/CD pipelines.

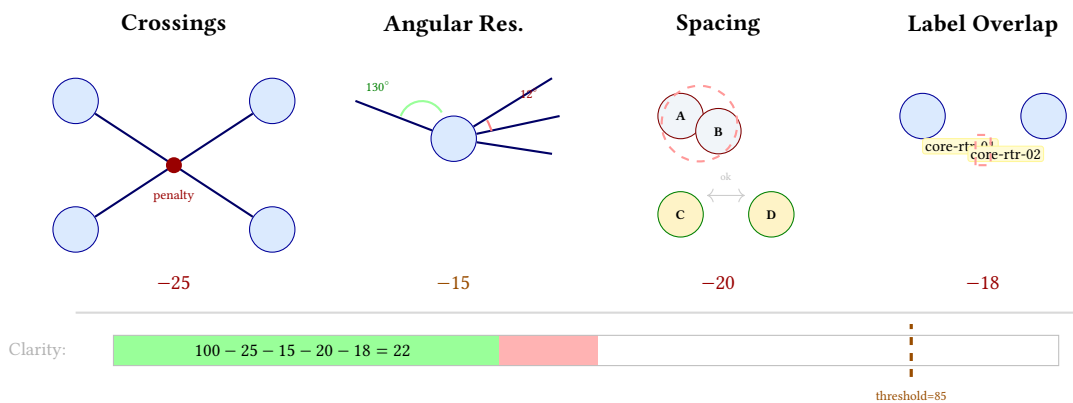


Figure 57. Clarity score: four quality factors illustrated spatially. Each panel shows the visual defect and its penalty. Edge crossings, narrow angular resolution, overlapping nodes, and colliding labels all subtract from 100. CI/CD pipelines gate on a configurable threshold (default: 85; this example scores 22—a clear failure).

Quality baselines are stored in `benches/regression_baselines.json`; CI fails if any metric degrades by more than 20%.

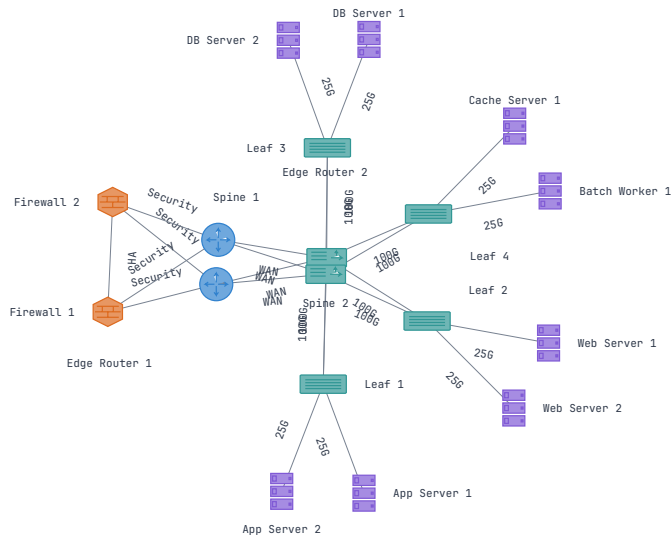


Figure 58. Quality metrics are computed alongside every render and can be overlaid on the output. Each node’s betweenness centrality, edge utilization, and label overlap are directly visible.

Separate from aesthetic quality, the validation module performs structural topology checks: isolated nodes, disconnected subgraphs, duplicate edge IDs, and missing labels. Results appear as Warning-severity entries in the `DiagnosticsEnvelope`.

25 Rendering Pipeline

25.1 SVG Renderer

The primary renderer produces SVG using the `svg crate`. Elements render in painter’s-model order: background map layers → isometric layer planes → group boundaries → edges → nodes → labels → effects.

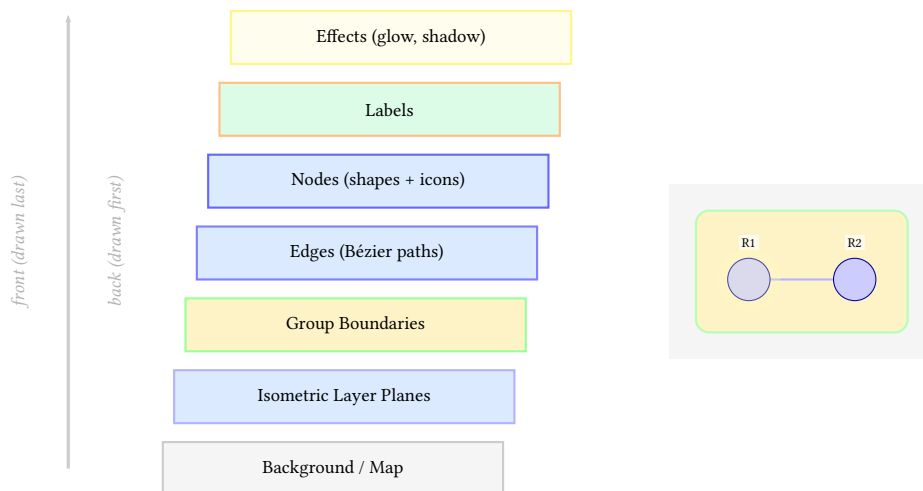


Figure 59. SVG rendering layer order (painter’s model). Elements are drawn back-to-front: background first, effects last. This ensures labels are always readable over edges, nodes always cover edge strokes at endpoints, and effect overlays (glow, shadow) render above all content.

25.2 Level of Detail

LOD suppresses visual details at configurable thresholds, following Abello et al. [17]:

Profile	Labels	Edge Lbl.	Interfaces	Effects
Conservative	2000	1000	1000	500
Balanced	500	300	300	200
Aggressive	200	100	100	50

Table 9. LOD suppression thresholds.

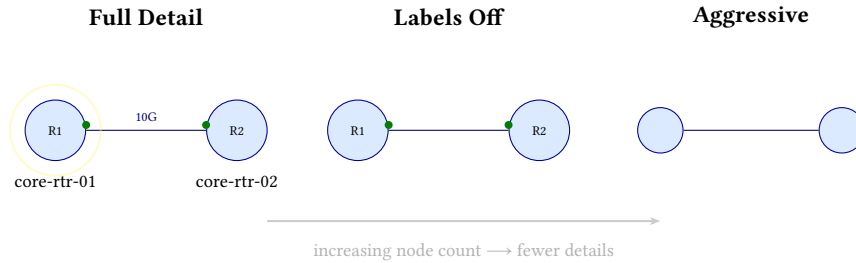


Figure 60. Level of detail (LOD) progressive suppression. As topology size grows past profile thresholds, labels, edge labels, interface markers, and visual effects are progressively hidden to maintain render performance and visual clarity.

25.3 Terminal Renderer

`netvis show` renders topology to Unicode box-drawing and ANSI color codes without a graphical display. `light` produces plain ASCII; `dark` produces ANSI-colored output.

25.4 Export Formats

SVG is the canonical output. Additional formats: PNG via `resvg`; PDF via `svg2pdf`; self-contained interactive HTML (WASM embedded as base64, pan/zoom/drag, openable from `file:///`); Graphviz DOT for interoperability.

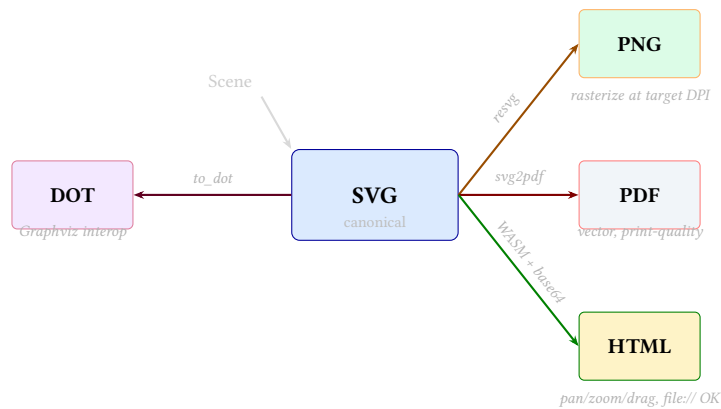


Figure 61. Export format pipeline. SVG is the canonical intermediate representation. `resvg` rasterizes to PNG at configurable DPI; `svg2pdf` produces vector PDF for print; the HTML exporter embeds WASM as base64 for interactive pan/zoom/drag openable from `file:///`; DOT export enables Graphviz interoperability.

26 Pluggable Node Renderer

The `NodeRenderer` trait defines the interface for custom node renderers. Built-in types (router, switch, server, firewall, cloud, `l3_switch`, and the full icon library from §8) are registered at startup via the inventory crate’s compile-time registration. The `render::registry` module provides lookup functions (`find_node_renderer`); unknown `node_type` values fall back to the default circle renderer.

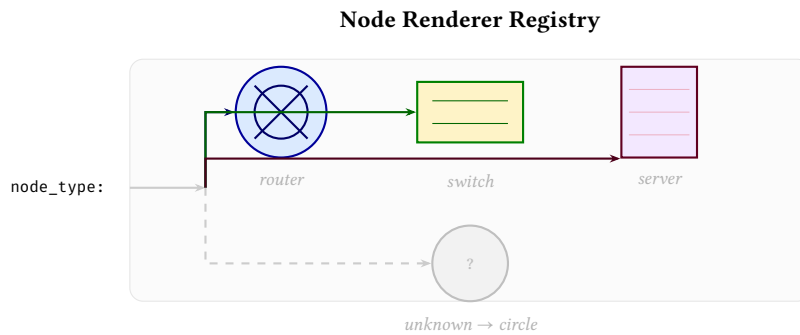


Figure 62. Pluggable node renderer dispatch. The `render::registry` module maps `node_type` strings to shape renderers. Built-in types (`router`, `switch`, `server`, `firewall`, `cloud`, `l3_switch`) are registered at startup. Unknown types fall back to the default circle renderer, ensuring graceful degradation for unrecognized device types.

27 API and Usage

27.1 Rust API

Function	Input	Returns
<code>contracts::v1::render</code>	Graph, Options	RenderEnvelope
<code>Layout::layout</code>	Graph	Scene
<code>SvgRenderer::render</code>	Scene	SVG string
<code>render_to_png</code>	SVG, PngOptions	PNG bytes
<code>render_to_pdf</code>	SVG, PdfOptions	PDF bytes
<code>to_dot</code>	NetVisGraph	DOT string
<code>ExportBuilder::to_png</code>	PngOptions	PNG bytes

Table 10. Core Rust API entry points.

27.2 WASM / JavaScript

JavaScript integration

```
import { init, renderToSVG, renderWithDiagnostics } from '@netvis/core';
await init();
const { output, diagnostics } = await renderWithDiagnostics(topologyJson);
```

A background Web Worker API offloads layout computation, streaming intermediate node positions via progress callbacks.

28 Deployment

NetVis compiles from a single Rust codebase to three targets:

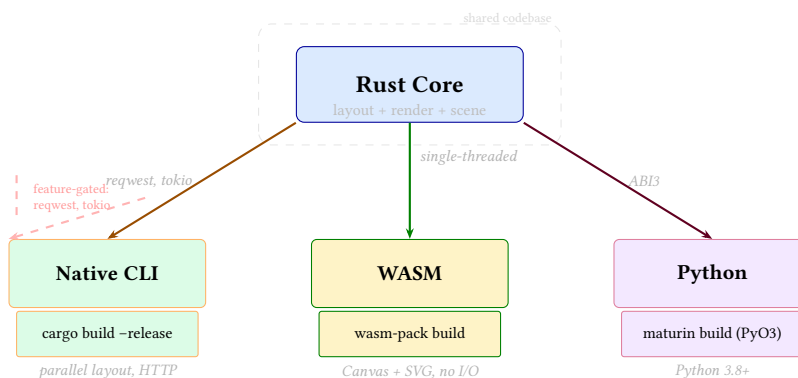


Figure 63. Deployment architecture. A single Rust codebase compiles to three targets via feature flags. WASM-incompatible crates (`reqwest`, `tokio`) are gated to native-only. The Python binding uses PyO3 with ABI3 stability for broad version support.

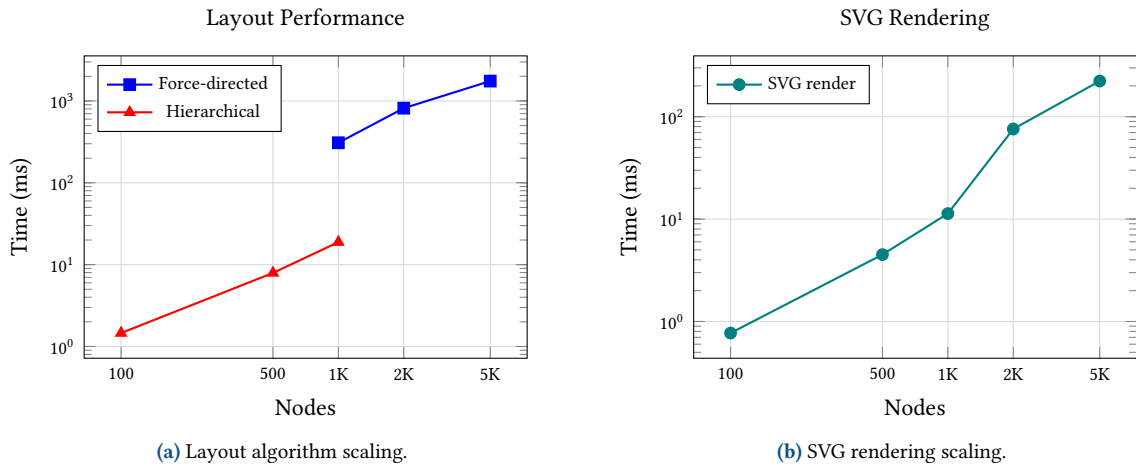


Figure 64. NetVis benchmark results on spine-leaf topologies. Left: layout computation time for force-directed (Barnes–Hut) and hierarchical (Sugiyama) engines. Right: SVG rendering time including label placement. Both axes are log-scaled; data points are median wall-clock times.

Target	Build command	Notes
Native CLI	<code>cargo build --release --features cli</code>	Full feature set; HTTP adapters; async I/O
WASM [18]	<code>wasm-pack build --target web</code>	Single-threaded; Canvas + SVG rendering; no file I/O
Python	<code>maturin build --features python</code>	PyO3, ABI3 stability (Python 3.8+)

Table 11. Deployment targets and build commands.

WASM-incompatible crates (`reqwest`, `tokio`) are gated to native-only via feature flags. Deterministic mode (CLI: `--deterministic`; YAML: `render.seed`) ensures reproducible output for CI/CD pipelines.

29 Performance

Benchmarks were measured on an Apple M4 Pro with 24 GB RAM, running macOS and Rust 1.93.

Operation	100	500	1K	2K	5K
Force-directed layout	20 ms	144 ms	309 ms	816 ms	1.75 s
Hierarchical layout	1.5 ms	8 ms	19 ms	—	—
SVG render	0.8 ms	4.5 ms	11 ms	76 ms	223 ms
Peak heap allocation	215 KB	952 KB	1.9 MB	3.8 MB	9.7 MB

Table 12. Performance baselines by topology size (spine-leaf topologies).

A 10,000-node spine-leaf topology (Enterprise engine, auto LOD) renders in 5.3 s end-to-end.

Tool	100 nodes	1K nodes	2K nodes
Graphviz <code>sfdp</code>	84 ms	250 ms	914 ms
Graphviz <code>neato</code>	88 ms	1,024 ms	4,434 ms
NetVis (full pipeline)	24 ms	330 ms	620 ms

Table 13. Comparative benchmark on identical spine-leaf topologies. NetVis times include layout, label placement, styling, and SVG rendering.

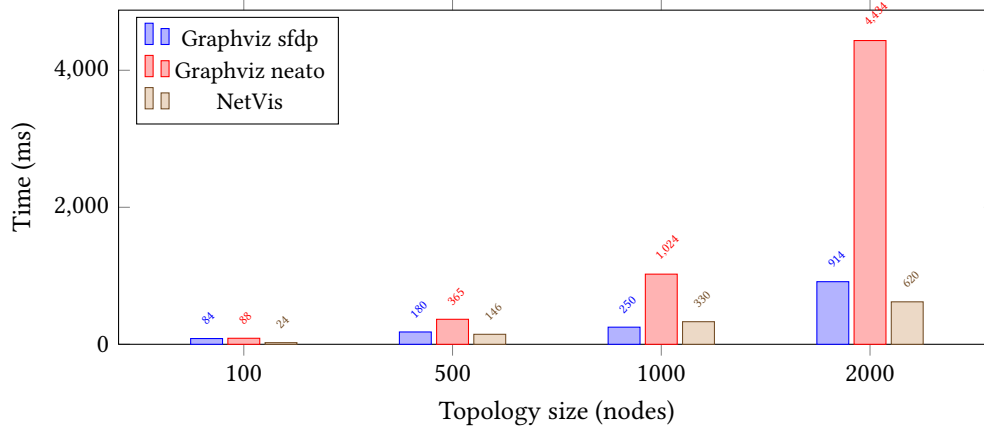


Figure 65. Comparative performance on identical spine-leaf topologies. NetVis includes the full pipeline (layout, labels, styling, SVG render); Graphviz includes layout and SVG output only.

30 Conclusion

This report has presented NetVis, a 90,000+ line Rust-based network topology visualization engine. Four contributions distinguish it from prior work:

1. **Dual-engine force-directed layout.** A Standard engine for topologies up to 2,000 nodes and a Barnes–Hut-accelerated Enterprise engine scaling to 10,000+ nodes (§5), complemented by six additional layout algorithms (hierarchical, geographic, isometric, orthogonal, subway, radial/tree).
2. **R*-tree-accelerated label placement.** A two-phase greedy algorithm using spatial indexing for $O(\log n)$ collision queries, placing labels for 5,000-node topologies in under 200 ms (§12).
3. **Composable refiner pipeline.** Ten post-layout refiners—label placement, edge bundling, smart routing, deoverlap, nudging, boundary bundling, and others—that compose in user-specified order to separate layout concerns from visual polish (§4, §14).
4. **Automated quality metrics.** Sixteen metric families (crossing count, angular resolution, overlap ratio, label clarity, and others) computed in-pipeline, enabling regression testing and objective comparison (§24).

Benchmarks on identical spine-leaf topologies show NetVis to be 1.5–7× faster than Graphviz sfdp/neato while producing the full rendering pipeline (layout, labels, styling, SVG output) rather than layout alone (§29). The system compiles from a single codebase to native CLI, WebAssembly, and Python targets (§28), and exposes domain-specific features—parallel edge rendering with automatic LAG aggregation (§7), interface rendering (§9), topology diffing (§15), traffic animation (§17), and metric encoding (§11)—that general-purpose graph tools lack.

31 Limitations

Limitation	Description	Workaround / Plan
Scale (>10K nodes)	Enterprise engine handles 10K nodes in ~5 s; 50K+ would benefit from GPU acceleration	Use hierarchical/geographic/tree layout; GPU via WebGPU is under investigation
Greedy label placement	Sequential placement; pathological dense clusters may produce overlaps	Suppress labels via LOD; increase canvas size; simulated-annealing post-pass planned
Single-threaded WASM	No threading in browser; large layouts slower	Use Web Workers for background computation; WASM threads pending
Non-determinism	HashMap iteration order is non-deterministic by default	Use <code>--deterministic</code> or <code>render.seed</code>
Adapter coverage	Only YAML, NetBox, LLDP/CDP built in	Adapter trait is public; community adapters encouraged
No user study	Quality claims based on automated metrics only	Controlled user study planned

Table 14. Known limitations and workarounds.

32 Future Work

1. **GPU-accelerated layout.** WebGPU compute shaders for 50,000+ node topologies.
2. **Streaming topology ingestion.** Live gNMI/gRPC streaming extending the timeline module (§16) to real-time updates.
3. **Simulated-annealing label post-pass.** Global optimization resolving remaining overlaps in dense clusters.
4. **User study.** Controlled perceptual evaluation with network engineers.
5. **Collaborative editing.** CRDT-based multi-user conflict resolution extending the editor module (§19).
6. **SNMP and streaming telemetry adapters.** Community adapters for SNMP, Napalm, and gNMI.
7. **Parallel edge animation.** Per-link traffic flow animation within a LAG bundle.

References

- [1] NetBox Labs, *NetBox: The leading solution for network automation*, 2024. [Online]. Available: <https://netbox.dev>
- [2] E. R. Gansner and S. C. North, “An open graph visualization system and its applications to software engineering,” 11, vol. 30, 2000, pp. 1203–1233.
- [3] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, “The open graph drawing framework (OGDF),” *Handbook of Graph Drawing and Visualization*, pp. 543–569, 2013.
- [4] M. Bostock, V. Ogievetsky, and J. Heer, “D3: Data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.
- [5] yWorks GmbH, *yEd graph editor*, 2024. [Online]. Available: <https://www.yworks.com/products/yed>
- [6] bluss, *Petgraph – graph data structure library for Rust*, 2024. [Online]. Available: <https://crates.io/crates/petgraph>
- [7] K. Sugiyama, S. Tagawa, and M. Toda, “Methods for visual understanding of hierarchical system structures,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.
- [8] T. M. J. Fruchterman and E. M. Reingold, “Graph drawing by force-directed placement,” *Software – Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991.
- [9] L. Verlet, “Computer “experiments” on classical fluids. I. thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, pp. 98–103, 1967.
- [10] J. Barnes and P. Hut, “A hierarchical $O(N \log N)$ force-calculation algorithm,” *Nature*, vol. 324, pp. 446–449, 1986.
- [11] J. Christensen, J. Marks, and S. Shieber, “An empirical study of algorithms for point-feature label placement,” in *ACM Transactions on Graphics*, vol. 14, 1995, pp. 203–232.
- [12] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, “The R*-tree: An efficient and robust access method for points and rectangles,” *ACM SIGMOD Record*, vol. 19, no. 2, pp. 322–331, 1990.
- [13] Stoeoef, *Rstar – R*-tree spatial index for Rust*, 2024. [Online]. Available: <https://crates.io/crates/rstar>
- [14] D. Holten and J. J. van Wijk, “Force-directed edge bundling for graph visualization,” *Computer Graphics Forum*, vol. 28, no. 3, pp. 983–990, 2009.
- [15] H. C. Purchase, “Metrics for graph drawing aesthetics,” *Journal of Visual Languages and Computing*, vol. 13, no. 5, pp. 501–516, 2002.
- [16] C. Dunne and B. Shneiderman, “Motif simplification: Improving network visualization readability with fan, connector, and clique glyphs,” *Proc. ACM CHI*, pp. 3247–3256, 2015.
- [17] J. Abello, F. van Ham, and N. Krishnan, “ASK-GraphView: A large scale graph visualization system,” in *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, 2006, pp. 669–676.
- [18] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

33 YAML Configuration Examples

This section consolidates the detailed YAML configuration snippets referenced throughout the document, keeping the core architectural discussion concise.

33.1 Basic Campus Network (§2)

The minimal topology YAML requires only name, nodes, and edges. The rendering section is optional; NetVis selects a suitable layout automatically.

Minimal topology (campus-edge.yaml)

```
name: Campus edge
render:
  theme: light
nodes:
  - { id: inet, label: Internet, node_type: cloud }
  - { id: fw1, label: Firewall, node_type: firewall }
  - { id: r1, label: Edge Router, node_type: router }
  - { id: s1, label: Access Switch, node_type: switch }
edges:
  - { from: inet, to: fw1, label: uplink }
  - { from: fw1, to: r1, label: transit }
  - { from: r1, to: s1, label: 10G }
```

```
netvis -input campus-edge.yaml -o campus.svg
```

33.2 Geographic CDN (§2)

Nodes with lat/lon attributes use the geographic layout with great-circle arc edges and a background map layer.

Geographic layout (cdn.yaml, excerpt)

```
render:
  preset: geographic
  theme: dark
geographic:
  show_map: true
  great_circle_arcs: true
nodes:
  - { id: nyc, label: New York, node_type: router,
      lat: 40.7128, lon: -74.0060 }
  - { id: lon, label: London, node_type: router,
      lat: 51.5074, lon: -0.1278 }
```

33.3 Isometric Multi-Layer (§2)

The isometric layout renders protocol stacks or layered architectures with a pseudo-3D perspective.

Isometric layout (multilayer.yaml, excerpt)

```
render:
  layout: isometric
layers:
  - { id: physical, label: Physical, order: 0 }
  - { id: network, label: Network, order: 1 }
  - { id: routing, label: Routing, order: 2 }
nodes:
  - { id: p-core1, label: Core-1, node_type: router,
      attributes: { layer: physical } }
```

33.4 Parallel Edges and Bond Groups (§2)

Multiple physical links between the same pair of nodes are grouped into an EdgeGroup.

Parallel edge modes

```
edges:
  # LAG: 4x 10G auto-aggregated to 40G
  - { from: core1, to: core2, label: "10G",
      bond_group: core-lag, parallel_mode: bundled }
  - { from: core1, to: core2, label: "10G",
      bond_group: core-lag }
  # Redundant uplinks shown as offset lines
  - { from: core1, to: dist1, label: "10G",
```

```
parallel_mode: offset }
- { from: core1, to: dist1, label: "10G" }
```

33.5 Subway Layout (§2)

Subway layout

```
render:
  layout: subway
  theme: blueprint
```

33.6 Pipeline Composition (§4)

Pipeline composition in YAML

```
render:
  layout: force_directed
  refiners: [labels, bundling, nudging, constraints]
```

33.7 Interface Configuration (§9)

Interface configuration

```
nodes:
- id: core-sw
  node_type: switch
  interfaces:
    - { id: eth0, status: up, mode: port, marker: dot }
    - { id: eth1, status: warning, mode: detailed, speed: 10G }
    - { id: eth2, status: critical, mode: port }
```

33.8 Edge Gradients (§10)

Edge gradient

```
edges:
- from: core1
  to: edge1
  gradient:
    start_color: "#3b82f6" # blue at source
    end_color: "#ef4444" # red at destination
```

33.9 Metric Encoding (§11)

Metric encoding

```
render:
  metric_encoding:
    - { attribute: cpu_utilization, channel: fill_color, palette: reds }
    - { attribute: packet_loss, channel: size,
        min_size: 15, max_size: 40 }
```

33.10 Traffic Animation (§17)

Traffic overlay

```
traffic:
edges:
- { id: core-to-agg, utilization: 0.75,
    direction: bidirectional, style: dot }
```

33.11 Annotations (§18)

Annotation configuration

```
annotations:
- anchor: router-fra-01
  type: callout
  text: "Primary BGP peer"
  shape: box
  layer: bgp-overlay
- anchor: rack-a-2
  type: group
  text: "Storage Cluster"
  style: dashed
```

33.12 Assigning Effects (§20)

Assigning effects

```
nodes:
- id: core-router
  highlight: true # default glow
- id: alert-node
  highlight: "emphasis" # glow-emphasis
```

References

- [1] NetBox Labs, *NetBox: The leading solution for network automation*, 2024. [Online]. Available: <https://netbox.dev>
- [2] E. R. Gansner and S. C. North, "An open graph visualization system and its applications to software engineering," 11, vol. 30, 2000, pp. 1203–1233.
- [3] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, "The open graph drawing framework (OGDF)," *Handbook of Graph Drawing and Visualization*, pp. 543–569, 2013.
- [4] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-driven documents," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.
- [5] yWorks GmbH, *yEd graph editor*, 2024. [Online]. Available: <https://www.yworks.com/products/yed>
- [6] bluss, *Petgraph – graph data structure library for Rust*, 2024. [Online]. Available: <https://crates.io/crates/petgraph>
- [7] K. Sugiyama, S. Tagawa, and M. Toda, "Methods for visual understanding of hierarchical system structures," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.
- [8] T. M. J. Fruchterman and E. M. Reingold, "Graph drawing by force-directed placement," *Software – Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991.
- [9] L. Verlet, "Computer "experiments" on classical fluids. I. thermodynamical properties of Lennard-Jones molecules," *Physical Review*, vol. 159, pp. 98–103, 1967.
- [10] J. Barnes and P. Hut, "A hierarchical $O(N \log N)$ force-calculation algorithm," *Nature*, vol. 324, pp. 446–449, 1986.
- [11] J. Christensen, J. Marks, and S. Shieber, "An empirical study of algorithms for point-feature label placement," in *ACM Transactions on Graphics*, vol. 14, 1995, pp. 203–232.
- [12] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, "The R*-tree: An efficient and robust access method for points and rectangles," *ACM SIGMOD Record*, vol. 19, no. 2, pp. 322–331, 1990.
- [13] Stoeoef, *Rstar – R*-tree spatial index for Rust*, 2024. [Online]. Available: <https://crates.io/crates/rstar>
- [14] D. Holten and J. J. van Wijk, "Force-directed edge bundling for graph visualization," *Computer Graphics Forum*, vol. 28, no. 3, pp. 983–990, 2009.
- [15] H. C. Purchase, "Metrics for graph drawing aesthetics," *Journal of Visual Languages and Computing*, vol. 13, no. 5, pp. 501–516, 2002.
- [16] C. Dunne and B. Shneiderman, "Motif simplification: Improving network visualization readability with fan, connector, and clique glyphs," *Proc. ACM CHI*, pp. 3247–3256, 2015.
- [17] J. Abello, F. van Ham, and N. Krishnan, "ASK-GraphView: A large scale graph visualization system," in *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, 2006, pp. 669–676.
- [18] A. Haas et al., "Bringing the web up to speed with WebAssembly," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

Index

adapters, 8
AestheticScore, 28
annotation, 25
API, 32
architecture, 8

bandwidth aggregation, 14
Barnes-Hut, 9
benchmarks, 33
betweenness centrality, 27
bond group, 13

callout, 25
CLI, 32
collision avoidance, 19
color palettes, 18
conclusion, 34
crossings, 28

data model, 6
deployment, 32
device icons, 14
diff, 24
drop shadow, 26

edge, 6
edge bundling, 20
edge gradients, 17
editor, 25

FDEB, 20
filter DSL, 27
flow direction, 17
force-directed edge bundling, 20
future work, 35
fuzzy matching, 28

glow, 26
GPU acceleration, 35
graph analytics, 27

HTML export, 30

interface status, 16
interfaces, 16

label placement, 19
LAG, 13
layout
 composite, 9
 force-directed, 9
 geographic, 9
 hierarchical, 9
 isometric, 9
 orthogonal, 9, 11
 radial, 9
 subway, 9, 11
 tree, 9
limitations, 34
link aggregation, 13
metric encoding, 18

node, 6
node renderer, 31
node shapes, 14
node_type, 14
nudging, 22

octilinear, 11
overlap, 28

parallel edges, 13
path emphasis, 17
performance, 33
pipeline, 8
ports, 16
Python binding, 32

quality metrics, 28

R*-tree, 19
refiners, 22
renderer, 8
rendering pipeline, 30
routing
 orthogonal, 22
 port-aware, 22
 smart, 22

scalability, 34
scene graph, 6
snapshot, 24
Sugiyama, 9
SVG, 30

temporal analysis, 24
themes, 12
timeline, 24
topology diffing, 24
topology graph, 6
traffic animation, 25

undo/redo, 25
utilization, 25

validation, 28
visual effects, 26

WASM, 25, 32
WASM API, 32
WebAssembly, 32

Revision History

Version	Date	Changes
1.5.0	Mar 2026	Subway, tree, radial layouts; interface rendering; node shapes; path emphasis; metric encoding; edge gradients; 6 new refiners; terminal renderer
1.4.0	Mar 2026	Parallel edges (5 modes); LAG aggregation; orthogonal layout; clarity scoring; graph analytics; filter DSL; fuzzy validation
1.3.0	Mar 2026	Margin insights; cover diagrams; design rule primitives
1.2.0	Mar 2026	Architectural strategy; formalized type logic
1.1.0	Mar 2026	Routing refiners; topology diffing; timeline; traffic animation; annotations; editor; effects; 16 quality metrics
1.0.0	Jan 2025	Initial pipeline specification